



Libre-V (v1.0)

An Open-Source RISC-V SoC.

User Reference

Version: 1.0
Date: August 15, 2026
Status: Under Development

About this Document

This document serves as the reference manual for the Libre-V System-on-Chip (SoC). It describes the SoC's functional blocks, their configuration options, and their intended usage. The manual is intended for software developers and hardware designers working with Libre-V platform at any abstraction.

Release information

Version	Date	Changes
1.0	Aug 2026	Initial release.

Conventions used in this manual

- **Monospace** text indicates register names, field names, signal names, or code.
- *Italic* text indicates a value that you must supply.
- Bit ranges are written [hi:lo], for example [7:4].
- **Blue** panels highlight notes; **red** panels highlight warnings and cautions the user should be careful about.

Abbreviations

RF	Register File
RO	Read-Only Access
RW	Read-Write Access
WO	Write-Only Access (Reads are undefined/ have no meaning)

Notes

- The official ARM/AMBA specifications have replaced the master–slave terminology with manager–subordinate. However, some parts of the Libre-V were developed before this terminology change and the legacy master–slave nomenclature may appear in corresponding source code.
- The base implementation/specification refers to the SoC configuration that you get on cloning the repository. It is possible to configure the SoC to your needs using a Terminal User Interface. Note that this feature is still in beta testing stage and is not recommended for use without caution.

Contents

About this Document	i
Release Information	i
Conventions	i
Abbreviations	i
Notes	i
 I Architecture and Software View	 1
 1 Architecture	 3
1.1 Overview	3
1.2 System	5
 2 Pico Subsystem	 6
2.1 Pico Package	6
2.2 COMMCTRL	7
2.3 AHB Manager MUX and Bus	8
2.4 Clock and Reset Generator (CRG)	8
2.5 Peripheral Cluster	8
 3 Universal Asynchronous Receiver/Transmitter (UART)	 9
3.1 Overview	9
3.2 Using the UART	10
3.2.1 DLAB (Divisor Latch Access Bit)	10
3.2.2 Setting the operating Baud Rate	10
3.2.3 Read/Write Shared Addresses	10
3.2.4 FIFOs	10
3.3 Register Descriptions	11
3.3.1 Address Map Summary	19
 4 Timer Family	 20
4.1 Basic Timer (BTimer)	20
4.1.1 Counting and the Prescaler	20
4.1.2 Compare, Overflow, and Auto-reset	21

4.1.3	Address Map Summary	21
4.1.4	Register Descriptions	21
4.1.5	Interrupts	23
4.2	General Purpose Timer (GPTimer)	24
4.2.1	Using the GPTimer	24
4.2.2	Operating Modes	25
4.2.3	Register Descriptions	25
5	Libre Software and Control	73
5.1	PicoRV32 Interrupt Handler	73
II	SoC Integration	74
6	Pico Subsystem	76
6.1	PicoRV32 Core and Package	76
7	On Chip SRAM	77
7.1	Overview	77
7.2	AHB Controller	77
7.2.1	AHB-Lite Primer	78
7.2.2	Phase Sequencing and Timing	78
7.2.3	Byte Lanes and Write Enables	78
7.3	AXI Controller	78
7.3.1	AXI Addressing Primer	79
7.3.2	Implementation	79
7.4	Libremc	81
7.4.1	Usage	82
7.5	Summary	82

List of Figures

1.1	Libre-V system-on-chip concept.	3
1.2	Libre-V platform block view.	4
2.1	Pico subsystem block view.	6
3.1	UART module high level schematic.	9
4.1	Btimer control flow	21
4.2	General Purpose Timer block diagram.	24
7.1	AXI/AHB SRAM module general block view.	77
7.2	axi2sram controller detailed block view.	78

List of Tables

3.1	Optional Modem Control Signals available in the UART module.	10
-----	--	----

Part I

Architecture and Software View

This part is intended for audience intending to understand the platform's architecture, configure, and adopt it in their own system.

Architecture

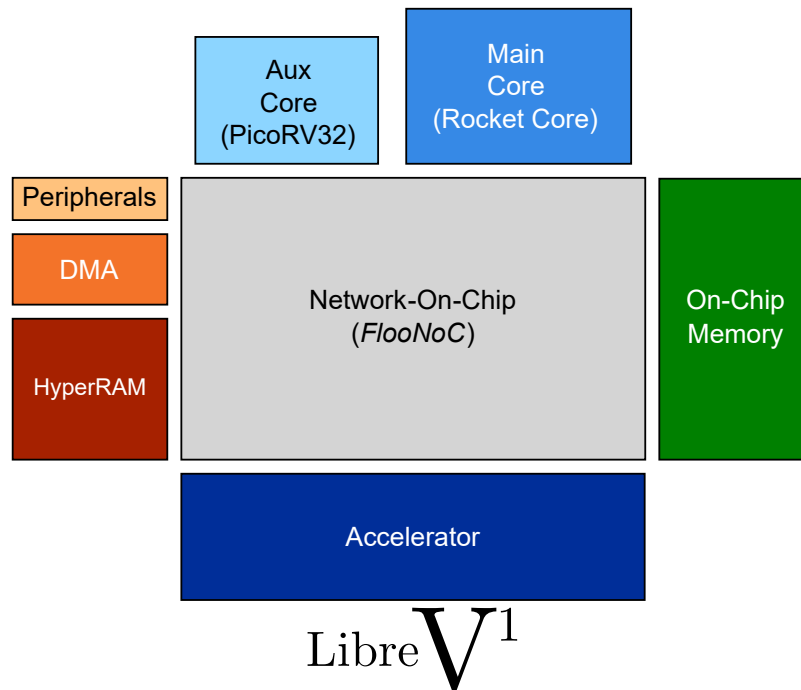


Figure 1.1: Libre-V system-on-chip concept.

1.1 Overview

Libre-V is a configurable system-on-chip platform intended to be used as host-CPU for testing ASICs, especially accelerators. Being composed of all open-source components, it is completely open source and licensed under the Apache-2.0 license¹. The platform uses a 32-bit auxiliary core coupled with a 64-bit main core. The auxiliary core implements RV32IMC instruction set, whereas the main core implements RV64GC instruction set.

This document does not discuss the implementation of either the RISC-V ISA nor any of the individual SoC components. For that, we direct the reader to the documentation available at [respective source\(s\)](#).

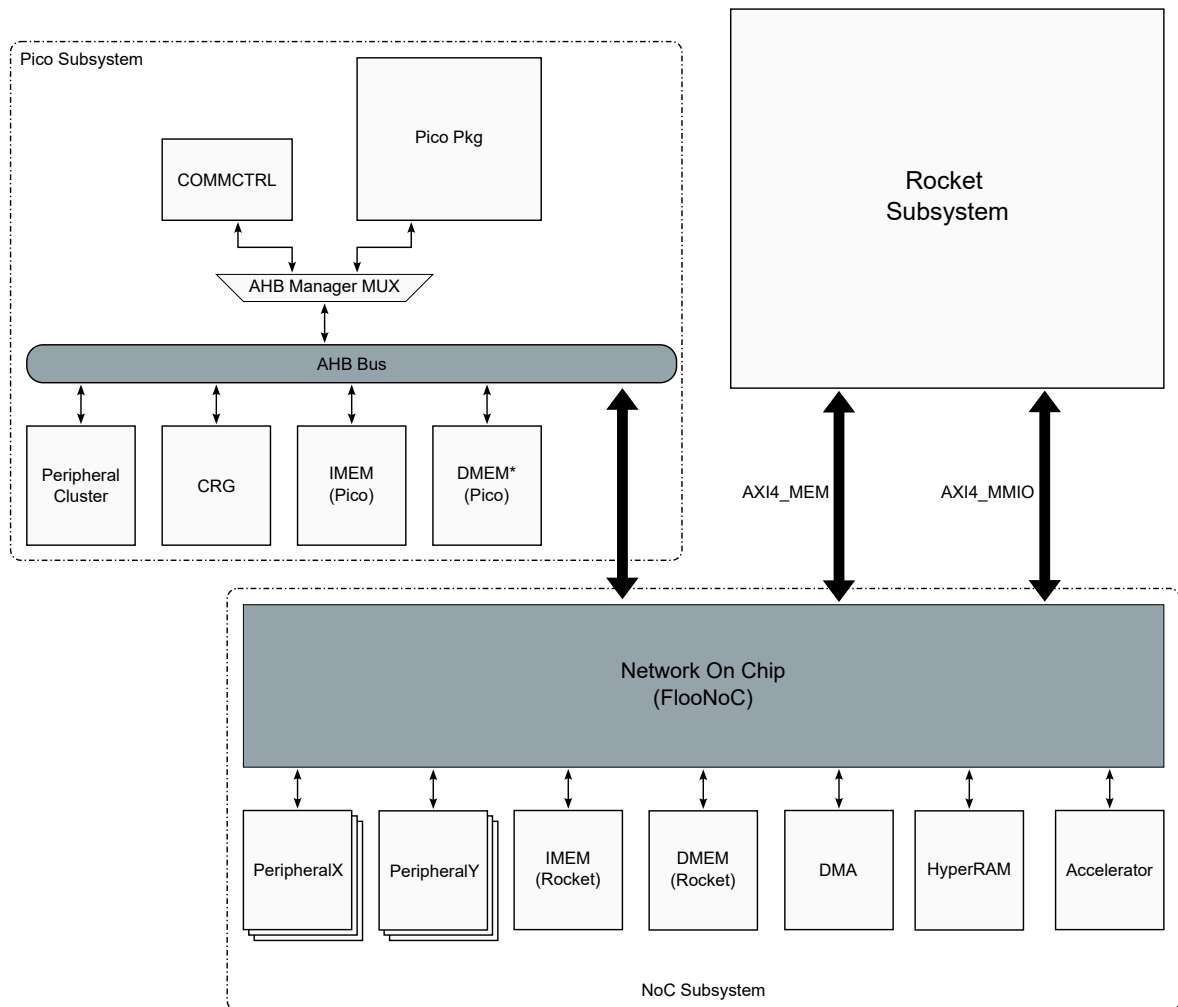


Figure 1.2: Libre-V platform block view.

1.2 System

Libre-V platform has 32-bit address space shared by memory-mapped I/O devices. It is composed of three subsystems:

1. Pico Subsystem
2. Rocket Subsystem
3. Network on Chip (NoC) Subsystem

See Figure 1.2 for a block-view of the platform. Note that the figure only shows two peripheral groups in the NoC subsystem. However, this is only meant for illustrations. It is possible to configure an Libre-V instance with more than 2 peripherals.

This part of the manual provides a software view of each component of the Libre-V:

- [Pico Subsystem](#)
- Rocket Subsystem
- [UART](#)
- [Timer](#)
- Libre Firmware and Device Drivers
- Libre Booting Process

¹Individual components may be protected by their own licensing. Refer the [repository](#) for more details.

Pico Subsystem

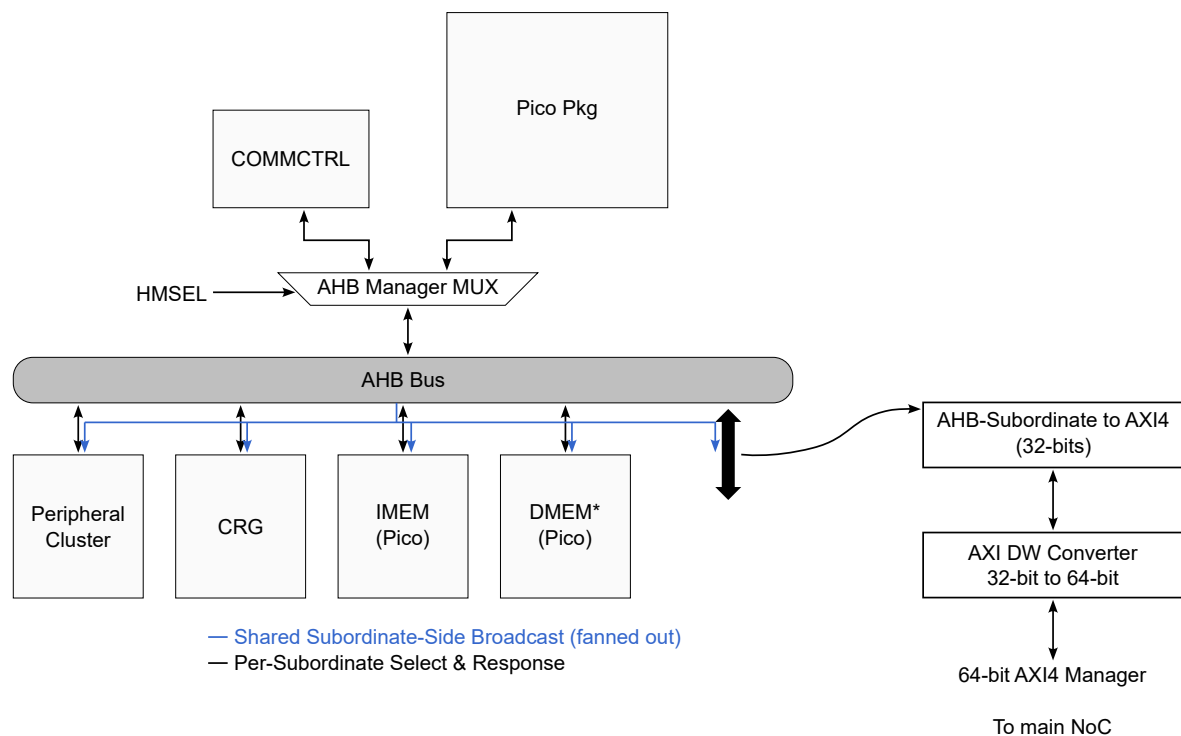


Figure 2.1: Pico subsystem block view.

The Pico subsystem is the SoC's control plane. It couples the PicoRV32 core (as [Pico Package](#)) with the host command interface, [COMMCTRL](#), the on-chip control registers ([CRG](#)), and the Pico-local peripherals ([Peripheral Cluster](#)), all joined by an [AHB Bus](#). It is the first block to wake at power-on and is responsible for loading and releasing the Rocket core ([??](#)). See [Figure 2.1](#).

2.1 Pico Package

PicoRV32 core is a lightweight, auxiliary core developed by Yosys. It implements RV32IMC instruction set. More generally, it can be configured as an RV32E, RV32I, RV32IC, RV32IM, RV32IMC core. It has the following features¹:

- Small, lightweight core requiring about 750 – 2000 LUTs in 7-Series Xilinx Architecture.
- High $f_{\max}$ of about 250 – 450 MHz on Xilinx FPGAs.
- Three options for interfaces: Native Memory (`picorv32`), AXI4-Lite Manager (`picorv32_axi`), or Wishbone (`picorv32_wb`).
- A built-in interrupt controller with optional, custom ISA support.
- An optional co-processor interface.

¹PPA results reproduced here from the repository without any additional verification.

- Configurable RF: It is possible to disable support for the registers x16 - x32, as well as RDCYCLE[H], RDTIME[H], and RDINSTRET[H] instructions². The RF can be configured as single or dual ported.

Relevant Link(s):

- [PicoRV32 Repository](#)
- [COMMCTRL/CHIPKIT Repository](#)
- [AMBA Bus Generator Repository](#)

The `pico_pkg_ahb` bridges the native-memory PicoRV32 core to the subsystem as a single AHB-Lite manager. The package attaches the core's private on-chip SRAM memory and fixes the boot convention:

Region	Base
.global, .text, .rodata, .data Stack	0x0000.0000

The reset vector is 0x0000.0000, and the initial stack pointer is placed at the top of the print-buffer region (0x0001.FC00). The host loads Pico firmware into IMEM through COMMCTRL before handover.

FIX THIS RATIO!

2.2 COMMCTRL

The COMMCTRL is a UART-based communication controller. It is capable of accessing i.e., reading and writing to any address in the 32-bit address space³. To use this module, UART interface is exposed over which binary form of the ASCII representation of the read and/or write commands is stream.

The format of a write command is as follows:

Byte	Field
0	ASCII (W or w)
1	single whitespace
2	0
3	ASCII(X or x)
4 - 11	WRADDR
12	single whitespace
13	0
14	ASCII(X/x)
15 - 22	WRDATA
23	CR
24	LF

and a read command is formatted as:

Byte	Field
0	ASCII (R or r)
1	single whitespace
2	0
3	ASCII(X or x)
4 - 11	RDADDR
12	CR
13	LF

²Refer to the official RV32E ISA.

³The address should have meaning of course, else it might cause an illegal access trap somewhere in the SoC or return a garbage value.

While the write command is 25 bytes large, the read command is shorter and is zero-padded to match this length. As shown in Figure 2.1, the COMMCTRL and Pico package are multiplexed as managers of the AHB-Lite bus. The control of this AHB MUX is handled by writing to the location 0x1000.0000. Particularly, writing a 1 to this location using the COMMCTRL hands over the control to the Pico package. Writing a 0 relinquishes the control back to the COMMCTRL.

Because it is a bus master, the host can load the Pico IMEM, load the Rocket memories across the NoC, and read results back.

Address	Symbol	Effect
0x1000.0000	COMMCTRL_HANDOVER_ADDR	Host writes 1 to hand bus ownership to the Pico.
0x1000.0004	COMMCTRL_IRQ_ADDR	Host write raises a one-cycle COMMCTRL interrupt.

2.3 AHB Manager MUX and Bus

The `pico_ahb.bus` is the subsystem interconnect. It arbitrates two managers, COMMCTRL and the Pico core, onto one AHB fabric and decodes the address to select a subordinate. Ownership is explicit: at power-on COMMCTRL holds the bus and loads memory; when the host writes the handover address (section 2.2), ownership passes to the Pico core, which then runs its firmware. Addresses that do not fall in a local subordinate are forwarded to the NoC through the AHB-to-AXI bridge (chapter 6).

The address space 0x0000.0000 – 0x2000.0000 is reserved for the managers of the PicoSys and includes the on-chip SRAM memory of the Pico core, the control registers defined by CRG, and the peripheral cluster.

2.4 Clock and Reset Generator (CRG)

2.5 Peripheral Cluster

Universal Asynchronous Receiver/Transmitter (UART)

3.1 Overview

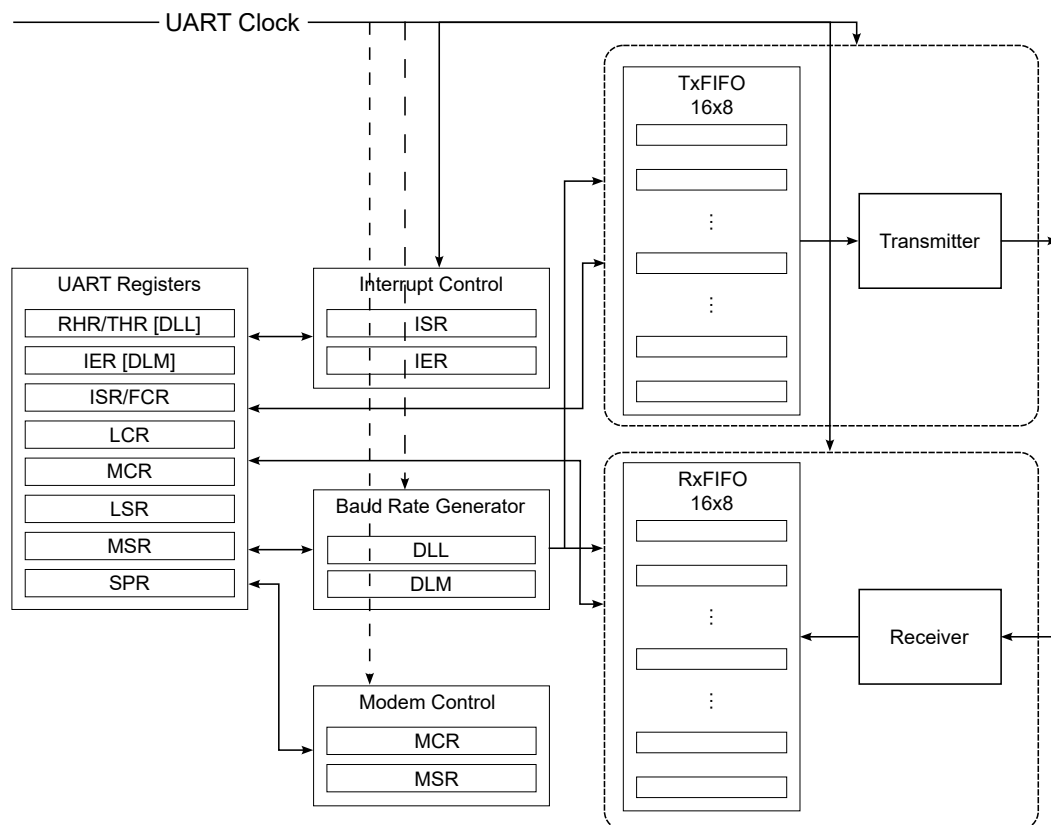


Figure 3.1: UART module high level schematic.

The configurator allows the user to add UART modules to either the peripheral cluster in the Pico subsystem or as a part of the NoC subsystem. It connects to the AHB bus over an AHB to APB bridge that might be shared with any other peripherals in the cluster.

Both modules provide optional modem control pins as listed in [Table 3.1](#). Use of most signals are implied by their names except, perhaps Ring Indicator and Carrier Detect input signals. These signals (all of them in the table above) are a part of the RS232 Interface specification. The two signals in particular are remnants of the dial-up internet/modem era, and can be safely ignored.

The modules translate the external AXI/APB interface to a register interface. Registers are byte-addressed on 4-byte boundaries. The 3-bit register index is taken from R/W address bits `addr[4:2]`:

$$\text{byte_offset} = \text{addr}[4:2] \times 4$$

The absolute base address is defined by the SoC interconnect (for example, `0x5000.0000` in Libre-V). Offsets in this document are relative to the UART instance base.

Fix the which UART is used for the NoC sys.

Signal	Function ¹
cts_n	Clear to Send (I)
dsr_n	Data Send Request (I)
ri_n	Ring Indicator (I)
cd_n	Carrier Detect (I)
rts_n	Ready to Send (O)
dtr_n	Data Ready (O)
out1_n	GP signal (optional, O)
out2_n	GP signal (optional, O)

Table 3.1: Optional Modem Control Signals available in the UART module.

3.2 Using the UART

3.2.1 DLAB (Divisor Latch Access Bit)

Bit 7 of the Line Control Register (**LCR.DLAB**) multiplexes the mapping at offsets 0x00 and 0x04:

- **DLAB = 0**: normal operation (data and interrupt enable registers).
- **DLAB = 1**: baud-rate divisor latches (**DLL**, **DLM**).

3.2.2 Setting the operating Baud Rate

To set the baud rate, find the baud rate divisor using:

$$\text{Divisor} = \frac{\text{SysClkFreq (in Hz)}}{16 \times \text{Baud Rate}}.$$

Having found the divisor, you need to quantize it to 16-bits². The design generates train of enable pulses based of the provided clock (as opposed to actually dividing the clock signal) based on this divisor.

To configure the UART, enable the divisor latches by setting **DLAB = 1**, and set the least significant byte of the 16-bit divisor in **DLL** and the most significant byte in **DLM**.

3.2.3 Read/Write Shared Addresses

Several register names share the same byte offset. The hardware distinguishes them by the bus transaction direction (a.we):

- A **read** at offset 0x00 returns **RHR**; a **write** stores **THR** (when **DLAB = 0**).
- A **read** at offset 0x08 returns **ISR**; a **write** updates **FCR**.

3.2.4 FIFOs

When **FCR.FIFO_EN** is set, the transmit and receive paths each use a 16-entry FIFO. When cleared, only a single-byte holding register is available on each path.

²The UART currently only supports integer divisors.

3.3 Register Descriptions

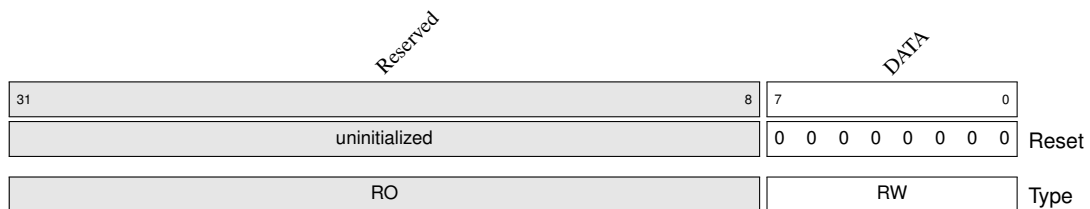
The following section describes the UART registers in the numerical order of address offset. This section will be useful for those wanting to write custom UART drivers or understand/debug existing drivers.

Register 0: Receiver Holding Register (RHR) / Transmitter Holding Register (THR)

Property	Value
Base address	SoC-defined
Byte offset	0x000
Reset value	0x00
Access type	RHR – RO; THR – WO

Read-sensitive (RHR). DLAB must be 0.

Register 3.1: UART DATA REGISTERS RHR/THR



Functional Description.

Offset 0x000 is shared by the Receiver Holding Register and the Transmitter Holding Register. A read returns the oldest received data byte from the receive FIFO (if enabled) or from the single-byte RHR. A write pushes a transmit data byte into the transmit FIFO (if enabled) or into the THR; transmission begins automatically when the transmitter is idle.

When DLAB = 1, this offset maps to the Divisor Latch LSB (DLL) instead; see Register 10.

Bit(s)	Name	Type	Reset	Description
7:0	DATA	RHR: RO THR: WO	0	Receive/transmit data byte. On read, returns the received character. On write, loads the next character to transmit.

Register 1: Interrupt Enable Register (IER)

Property	Value
Base address	SoC-defined
Byte offset	0x004
Reset value	0x00
Access type	RW

DLAB must be 0. When DLAB = 1, this offset maps to DLM.

Register 3.2: UART INTERRUPT ENABLE REGISTER (IER)

Reserved																MSTAT RLSTAT THRE DR			
																3	2	1	0
0																0	0	0	0
RW																RW	RW	RW	RW
																Type			

Functional Description.

The Interrupt Enable Register masks individual interrupt sources. A bit set to 1 allows the corresponding condition to contribute to the interrupt output `irq_o`. Reading or writing IER does not clear pending interrupts.

Bit(s)	Name	Type	Reset	Description
7:4	reserved	RO	0	Reserved. Read as 0. Writes are stored but have no effect.
3	MSTAT	RW	0	Modem Status interrupt enable. When 1, a change in modem status lines may assert an interrupt.
2	RLSTAT	RW	0	Receive Line Status interrupt enable. When 1, overrun, parity, framing, or break errors may assert an interrupt.
1	THRE	RW	0	Transmitter Holding Register Empty interrupt enable. When 1, an empty THR/TX FIFO may assert an interrupt.
0	DR	RW	0	Data Ready / Reception Timeout interrupt enable. When 1, received data available or a character timeout (FIFO mode) may assert an interrupt.

Register 2: Interrupt Status Register (ISR)

Property	Value
Base address	SoC-defined
Byte offset	0x008
Reset value	0xC1
Access type	RO

Read-only. DLAB must be 0.

Register 3.3: UART INTERRUPT STATUS REGISTER (ISR)

Reserved																FIFOS.EN Reserved ID STATUS			
																7	6	5	4
0																1	1	0	0
RO																RO	RO	RO	RO
																Type			

Functional Description.

The Interrupt Status Register reports whether an interrupt is pending and, if so, the highest priority interrupt source. The register is implemented in `obi_uart_interrupts.sv` and is not stored in the main register flip-

flop array. Reading ISR identifies the interrupt source but does not by itself clear all interrupt conditions; associated status registers (LSR, MSR, RHR) may need to be serviced.

Bit(s)	Name	Type	Reset	Description
7:6	FIFOS.EN	RO	2'b11	FIFO capability field. Hardwired to 2'b11 to indicate 16550A-style FIFO support.
5:4	reserved	RO	0	Reserved. Read as 0.
3:1	ID	RO	0	Interrupt identification code when STATUS = 0: • 011: Receive line status • 010: Received data available • 110: Character timeout (FIFO mode) • 001: THR empty • 000: Modem status
0	STATUS	RO	1	Interrupt status. 0 = interrupt pending; 1 = no interrupt pending.

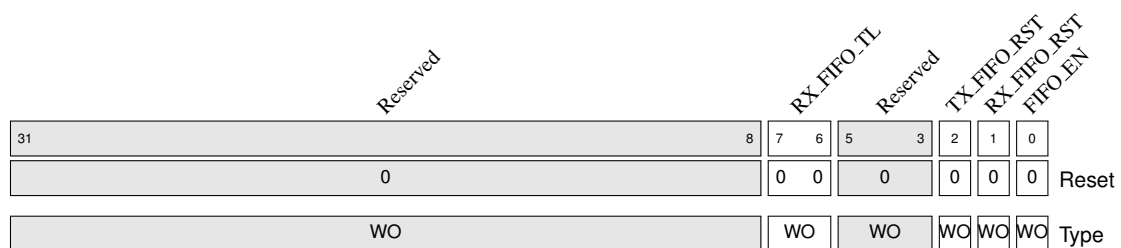
The ID field is updated in the priority encoder in the interrupt submodule.

Register 3: FIFO Control Register (FCR)

Property	Value
Base address	SoC-defined
Byte offset	0x008
Reset value	0x00
Access type	WO

Write-only. Shares offset with ISR.

Register 3.4: UART FIFO CONTROL REGISTER (FCR)



Functional Description.

The FIFO Control Register configures the on-chip transmit and receive FIFOs that are a part of the UART Tx and Rx submodules. Writes to this register at offset 0x008 update FCR; reads at the same offset return ISR.

The RX and TX FIFOs are each 16 entries deep. The RX FIFO stores 11 bits per entry (8 data bits plus break, framing, and parity error flags). The TX FIFO stores 8-bit characters.

Bit(s)	Name	Type	Reset	Description
7:6	RX_FIFO_TL	WO	0	Receive FIFO trigger level: • 00: 1 character • 01: 4 characters • 10: 8 characters • 11: 14 characters
5:3	reserved	WO	0	Reserved.
2	TX_FIFO_RST	WO	0	Write 1 to reset/clear the transmit FIFO.
1	RX_FIFO_RST	WO	0	Write 1 to reset/clear the receive FIFO.
0	FIFO_EN	WO	0	FIFO enable. 0 = 8250-style single-byte THR/RHR; 1 = 16550A 16-byte FIFO mode.

Register 4: Line Control Register (LCR)

Property	Value
Base address	SoC-defined
Byte offset	0x00C
Reset value	0x00
Access type	RW

Accessible regardless of DLAB.

Register 3.5: UART LINE CONTROL REGISTER (LCR)

Reserved								DLAB	SBREAK	SPAR	EPAR	PEN	STP	WLS	
31								7	6	5	4	3	2	1	0
0								0	0	0	0	0	0	0	0
RW								RW	RW	RW	RW	RW	RW	RW	Type
Reset															

Functional Description.

The Line Control Register defines the serial frame format and the DLAB address-multiplexing bit. Changes to word length, parity, and stop bits take effect for subsequently transmitted and received characters.

Bit(s)	Name	Type	Reset	Description
7	DLAB	RW	0	Divisor Latch Access Bit. When 1, offsets 0x00 and 0x04 access DLL and DLM. When 0, they access RHR/THR and IER.
6	SBREAK	RW	0	Set Break. When 1, forces the transmit output to a spacing (low) condition.
5	SPAR	RW	0	Stick Parity. When parity is enabled, forces parity bit to 0 (if EPAR = 1) or 1 (if EPAR = 0).
4	EPAR	RW	0	Even Parity select. 0 = odd parity; 1 = even parity (when parity enabled).
3	PEN	RW	0	Parity Enable. When 1, a parity bit is transmitted and checked.
2	STP	RW	0	Stop Bits. 0 = one stop bit; 1 = two stop bits (1.5 stop bits when word length is 5 bits).

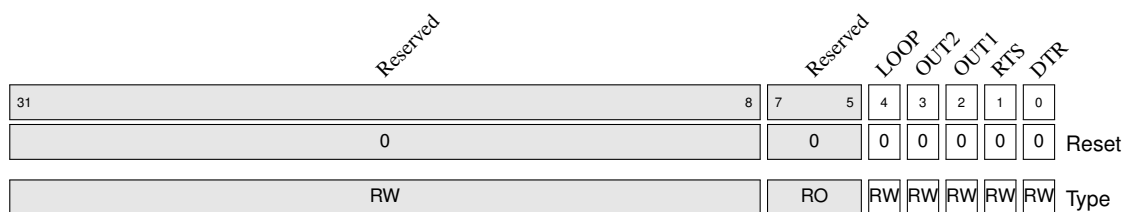
Bit(s)	Name	Type	Reset	Description
1:0	WLS	RW	0	Word Length Select: • 00: 5 data bits • 01: 6 data bits • 10: 7 data bits • 11: 8 data bits

Register 5: Modem Control Register (MCR)

Property	Value
Base address	SoC-defined
Byte offset	0x010
Reset value	0x00
Access type	RW

Accessible regardless of DLAB.

Register 3.6: UART MODEM CONTROL REGISTER (MCR)



Functional Description.

The Modem Control Register drives modem output pins and enables internal loopback for test. Modem outputs are active-low on the external pins (rts_no, dtr_no, etc.).

Bit(s)	Name	Type	Reset	Description
7:5	reserved	RO	0	Reserved. Read as 0.
4	LOOP	RW	0	Loopback. When 1, the transmitter output is looped back to the receiver input internally.
3	OUT2	RW	0	Auxiliary output 2 (optional GPIO).
2	OUT1	RW	0	Auxiliary output 1 (optional GPIO).
1	RTS	RW	0	Request To Send output.
0	DTR	RW	0	Data Terminal Ready output.

Register 6: Line Status Register (LSR)

Property	Value
Base address	SoC-defined
Byte offset	0x014
Reset value	0x60
Access type	RO

Read-sensitive. DLAB must be 0.

Register 3.7: UART LINE STATUS REGISTER (LSR)

Reserved								FIFO ERR	TXE	THRE	BI	FE	PE	OE	DR
31							8	7	6	5	4	3	2	1	0
0								0	1	1	0	0	0	0	0
RO								RO	RO	RO	RO	RO	RO	RO	RO

Reset

Type

Functional Description.

The Line Status Register provides transmit readiness and receive status, including error flags. Unlike some UART designs, error information is *not* embedded in the RHR read data; errors are reported only in LSR. Reading LSR clears the receive-line-status interrupt. Individual error bits are cleared on LSR read when no additional FIFO errors remain.

Bit(s)	Name	Type	Reset	Description
7	FIFO_ERR	RO	0	FIFO Error. Set when at least one character with an error was received into the RX FIFO.
6	TXE	RO	1	Transmitter Empty. 1 when the transmitter shift register and holding logic are completely idle.
5	THRE	RO	1	THR Empty. 1 when the THR or TX FIFO can accept a new write.
4	BI	RO	0	Break Indicator. 1 when a break condition was detected on the receive line.
3	FE	RO	0	Framing Error. 1 when the stop bit was not received as expected.
2	PE	RO	0	Parity Error. 1 when the received parity bit did not match.
1	OE	RO	0	Overrun Error. 1 when a new character arrived before the previous one was read (RHR mode) or when the RX FIFO was full (FIFO mode).
0	DR	RO	0	Data Ready. 1 when received data is available in RHR or the RX FIFO.

Register 7: Modem Status Register (MSR)

Property	Value
Base address	SoC-defined
Byte offset	0x018
Reset value	0x00
Access type	RO

Read-sensitive. DLAB must be 0.

Register 3.8: UART MODEM STATUS REGISTER (MSR)

Reserved								CD	RI	DSR	CTS	DCD	TER1	DDSR	DCTS
31							8	7	6	5	4	3	2	1	0
0								0	0	0	0	0	0	0	0
RO								RO	RO	RO	RO	RO	RO	RO	RO

Reset

Type

Register 9: Divisor Latch LSB (DLL)

Property	Value
Base address	SoC-defined
Byte offset	0x000
Reset value	0x01
Access type	RW

DLAB must be 1. Shares offset with RHR/THR.

Register 3.10: UART DIVISOR LATCH LSB (DLL)

Reserved																DIV LSB															
318																70															
uninitialized																00000001Reset															
RO																RWType															

Functional Description.

When DLAB = 1, offset 0x000 maps to the least significant byte of the 16-bit baud rate divisor. The divisor is combined with DLM as:

A divisor of 0 is invalid and disables baud-rate generation. Writing DLL or DLM resets the internal baud counters.

Bit(s)	Name	Type	Reset	Description
7:0	DIV_LSB	RW	0x01	Least significant byte of the baud divisor latch.

Register 10: Divisor Latch MSB (DLM)

Property	Value
Base address	SoC-defined
Byte offset	0x004
Reset value	0x00
Access type	RW

DLAB must be 1. Shares offset with IER.

Register 3.11: UART DIVISOR LATCH MSB (DLM)

Reserved																DIV_MSB								
318																70								
0																00000000								Reset
RW																RW								Type

Functional Description.

When DLAB = 1, offset 0x004 maps to the most significant byte of the baud rate divisor. See Register 9 for the baud-rate formula.

Bit(s)	Name	Type	Reset	Description
7:0	DIV_MSB	RW	0x00	Most significant byte of the baud divisor latch.

3.3.1 Address Map Summary

Offset	DLAB = 0 (read)	DLAB = 0 (write)	DLAB = 1 (read)	DLAB = 1 (write)
0x000	RHR	THR	DLL	DLL
0x004	IER	IER	DLM	DLM
0x008	ISR	FCR	ISR	FCR
0x00C	LCR	LCR	LCR	LCR
0x010	MCR	MCR	MCR	MCR
0x014	LSR	—	—	—
0x018	MSR	—	—	—
0x01C	SPR	SPR	SPR	SPR

Timer Family

Libre-V offers three different timer options:

1. [Basic Timer](#)
2. General Purpose Timer
3. Advanced Timer/PWM

These timer variants differ in their functionality. The feature set increases in the order listed above. All modules are memory mapped APB peripherals.

4.1 Basic Timer (BTimer)

The BTimer module is the APB timer from the PULP platform¹. It is a free-running 32-bit up-counter with a programmable clock prescaler and a compare register, and it raises two physical² interrupts: a counter *overflow* and a *compare match*. Note that the BTimer module can instantiate several timer cores behind an internal APB address mux (selected by PADDR).

The Btimer core exposes three 32-bit registers. They are byte-addressed on 4-byte boundaries; the register index is taken from APB address bits PADDR[3:2]:

$$\text{byte_offset} = \text{PADDR}[3:2] \times 4$$

The base address of a Btimer can be found in the generated software library file(s) as these depend on the user-specific configuration. Offsets in this document are relative to the timer instance base.

Using the Btimer

Figure [Figure 4.1](#) illustrates the control flow of the Btimer module.

4.1.1 Counting and the Prescaler

When **CTRL.EN** is set, the counter advances at a rate governed by the 3-bit prescaler field **CTRL.PSC**. A free-running *cycle counter* counts input clock cycles from 0 up to the prescaler value and then wraps:

- **PSC = 0** (normal mode): the counter increments on every clock cycle.
- **PSC $\neq$ 0** (prescaled mode): the counter increments once every $(\text{PSC} + 1)$ clock cycles.

So the effective counting rate is $f_{\text{clk}}/(\text{PSC} + 1)$.

¹<https://github.com/pulp-platform/apb.timer.git>

²By physical we mean that the interrupts are exposed as output ports as opposed to updating registers. These interrupts are made software-readable by connecting it to interrupt ports of the cores either pico or rocket.

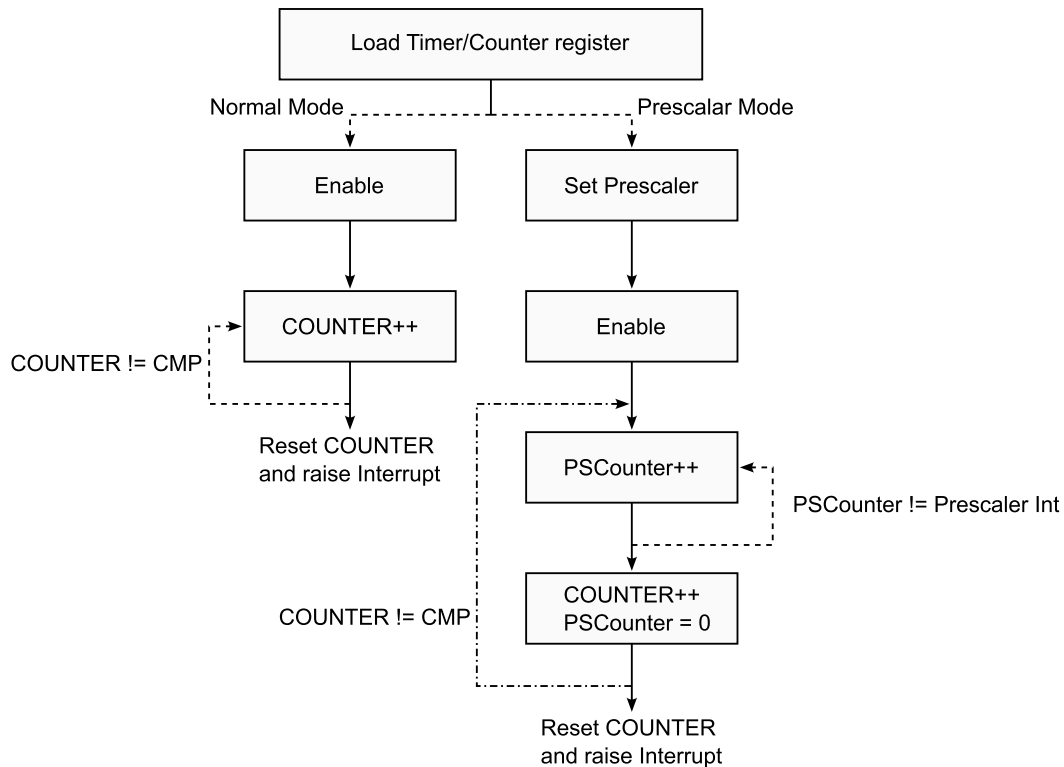


Figure 4.1: Btimer control flow

4.1.2 Compare, Overflow, and Auto-reset

The counter is compared against the **CMP** register on each tick:

- **Compare match:** when $CMP \neq 0$ and the counter equals CMP, the compare-match interrupt (`irq_o[1]`) is asserted and the counter is reset to 0. A CMP value of 0 disables the compare function.
- **Overflow:** when the counter reaches `0xFFFF_FFFF` and the next tick occurs, the overflow interrupt (`irq_o[0]`) is asserted and the counter wraps to 0.

Both interrupts are single-cycle pulses generated combinatorially on the tick. In addition, *writing* the CMP register resets the counter to 0, which makes it easy to start a fresh timed interval. This gives a straightforward periodic-tick usage: program a prescaler and a compare value, enable the timer, and take a compare-match interrupt every $(CMP + 1) \times (PSC + 1)$ clock cycles.

4.1.3 Address Map Summary

Offset	Register	Access
0x000	TIMER (counter)	RW
0x004	CTRL (control)	RW
0x008	CMP (compare)	RW

4.1.4 Register Descriptions

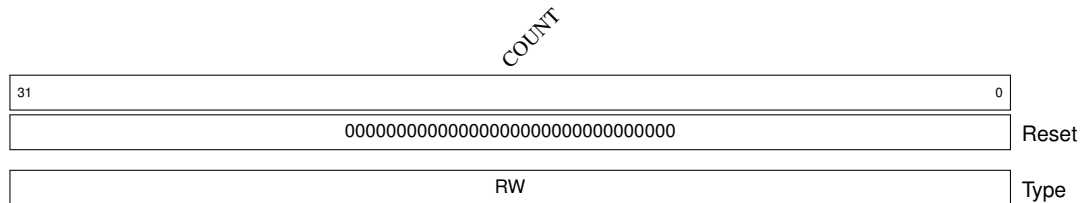
The following section describes the TIMER registers in the numerical order of address offset.

Register 0: Timer Counter Register (TIMER)

Property	Value
Base address	SoC-defined
Byte offset	0x000
Reset value	0x00
Access type	RW

Free-running count. Auto-resets on match/overflow/CMP write.

Register 4.1: TIMER COUNTER REGISTER (TIMER)



Functional Description.

Offset 0x000 holds the current 32-bit counter value. A read returns the instantaneous count. A write preloads the counter, allowing software to start from an arbitrary value. When the timer is enabled the hardware advances this register at the prescaled rate, and clears it to 0 on a compare match, on overflow, or when the **CMP** register is written.

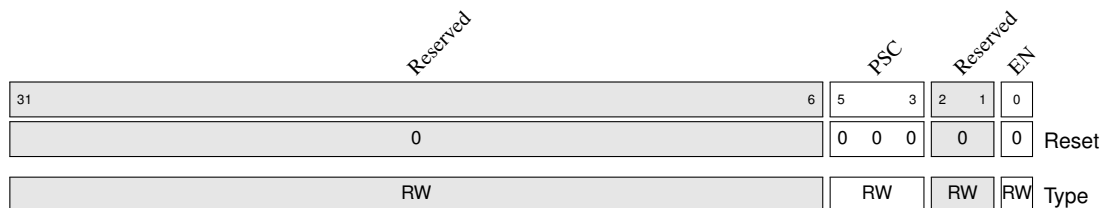
Bit(s)	Name	Type	Reset	Description
31:0	COUNT	RW	0	Current timer count. Readable at any time; a write preloads the counter. Advances at $f_{clk}/(PSC + 1)$ while CTRL.EN = 1.

Register 1: Timer Control Register (CTRL)

Property	Value
Base address	SoC-defined
Byte offset	0x004
Reset value	0x00
Access type	RW

Only EN (bit 0) and PSC (bits 5:3) are functional.

Register 4.2: TIMER CONTROL REGISTER (CTRL)



Functional Description.

The Control Register enables the counter and sets the prescaler. The full 32-bit word is stored, so the reserved bits are writable and read back their last written value, but they have no effect on operation.

Bit(s)	Name	Type	Reset	Description
31:6	reserved	RW	0	Reserved. Stored but have no effect.
5:3	PSC	RW	0	Prescaler. The counter increments once every (PSC + 1) clock cycles. PSC = 0 selects normal mode (increment every cycle).
2:1	reserved	RW	0	Reserved. Stored but have no effect.
0	EN	RW	0	Enable. When 1, the counter runs; when 0, the counter is frozen at its current value.

Register 2: Timer Compare Register (CMP)

Property	Value
Base address	SoC-defined
Byte offset	0x008
Reset value	0x00
Access type	RW

Writing CMP resets the counter to 0. CMP = 0 disables compare.

Register 4.3: TIMER COMPARE REGISTER (CMP)

CMP	
31	0
0	
RW	
	Reset
	Type

Functional Description.

Offset 0x008 holds the 32-bit compare value. When the counter reaches this value (and the value is non-zero), the compare-match interrupt `irq_o[1]` is asserted and the counter is reset to 0, producing a periodic tick of period $(CMP + 1) \times (PSC + 1)$ clock cycles. Writing this register also immediately resets the counter to 0, so software should program **CMP** before starting the count. A value of 0 disables the compare path, leaving only the overflow interrupt active.

Bit(s)	Name	Type	Reset	Description
31:0	CMP	RW	0	Compare value. On <code>COUNT == CMP</code> (with <code>CMP ≠ 0</code>) the compare-match interrupt fires and the counter clears. Writing this field also clears the counter. <code>CMP = 0</code> disables the compare match.

4.1.5 Interrupts

The timer core drives a 2-bit interrupt output `irq_o[1:0]`:

Bit	Name	Condition
0	Overflow	Counter wraps from 0xFFFF_FFFF.
1	Compare match	Counter equals a non-zero CMP.

4.2 General Purpose Timer (GPtimer)

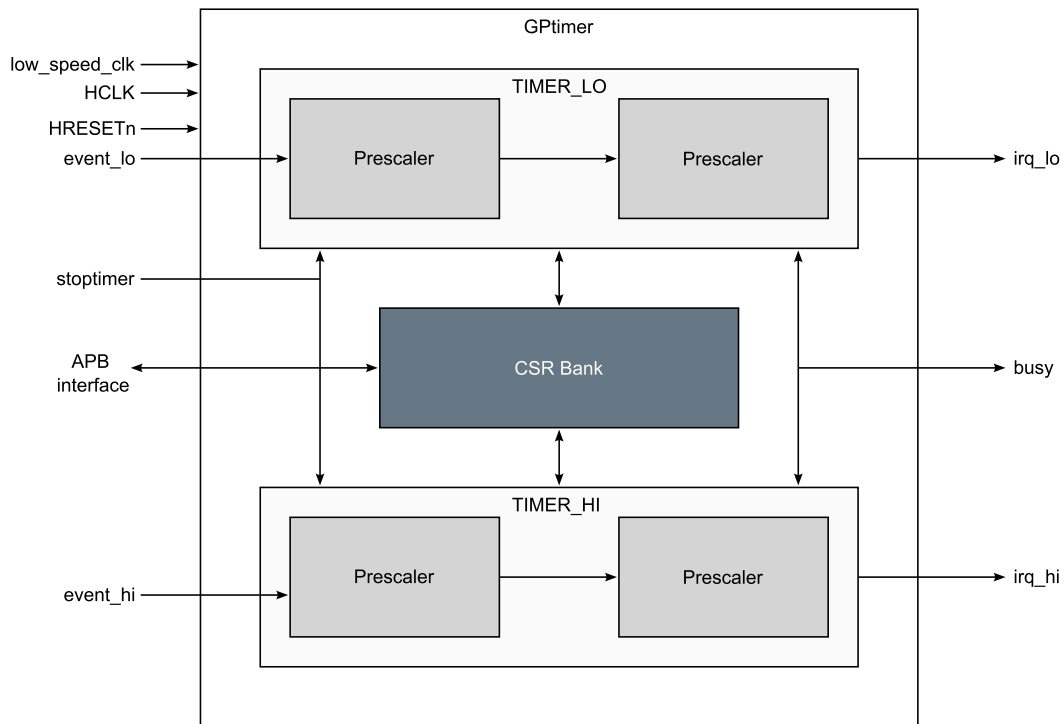


Figure 4.2: General Purpose Timer block diagram.

GPtimer is the more flexible APB Timer unit³ developed by PULP⁴. This part of the documentation is adapted⁵. Figure 4.2 illustrates a high level block diagram of the general purpose timer module.

Features

- Multiple trigger input sources.
- Two 32-bit configurable prescaler.
- Configurable interrupts.
- Support for two independent 32-bit timers or a single 64-bit timer.
- Supports one-shot and compare-clear modes.
- Can be used to implement the RISC-V machine timer (`mtime`, `mtimecmp` platform registers using the 64-bit timer with MTIME mode enabled.)

4.2.1 Using the GPtimer

GPtimer can either be configured as two 32-bit independent timers or a single 64-bit timer. It has two timers, `Timer_hi` and `Timer_lo`, which are responsible to generate respective interrupt signals. Working of both timers is functionally identical in the 32-bit mode (and similar to `Btimer`), whereas both are cascaded in 64-bit mode.

To extend the available range of timer periods, there is a secondary clock pin provided in addition to the system clock. Using the CSRs, one can select between the two clock sources. When used in 32-bit mode,

³https://github.com/openhwgroup/core-v-mcu/tree/master/rtl/apb_timer_unit.

⁴The source code is available as a part of the OpenHW's Core-V-MCU repository. A standalone directory of the GPtimer (i.e., apb timer unit) was not available).

⁵https://docs.openhwgroup.org/projects/core-v-mcu/doc-src/ip-blocks/apb_timer.html

both the timers may be independently and identically programmed and used as both timers have identical CSRs. To operate the timer in 64-bit mode, all CSRs of Timer_lo and the **VAL**, **CMP**, and **RESET** CSRs of the Timer_hi are used. Remaining CSRs of the Timer_hi are ignored.

4.2.2 Operating Modes

1. One Shot Mode

One-shot mode can be independently enabled for both LO and HI timer.

- In 32-bit mode, if the ONE_SHOT_BIT bitfield in the **CFG** CSR is set to 1, one-shot mode is enabled. The timer will automatically disable when the its counter reaches the value in **TIMER_CMP_*** for the first time. When it is disabled, the timer will pause its operation (i.e the timer counter and irq will retain its current value). **TIMER_*** counter and irq are reset to 0, when the reset is issued.
- In 64-bit mode, if the ONE_SHOT_BIT bitfield in the **CFG_REG_LO** CSR is set to 1, one-shot mode is enabled for the entire 64-bit timer. The cascaded timer will automatically disable when Timer_lo counter reaches **TIMER_CMP_LO**, and Timer_hi counter reaches **TIMER_CMP_HI** for the first time. When its disabled, the cascaded timer will pause its operation (i.e the timer counter and irq_lo will retain its current value).

2. Compare Clear Mode

Compare Clear mode can be independently enabled for both Timer_lo and Timer_hi.

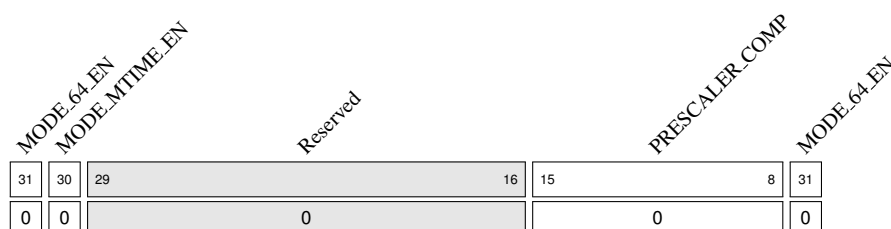
- In 32-bit mode, if the CMP_CLR_BIT bitfield in the **CFG** CSR is set to 1, Compare Clear mode is enabled for Timer_*. When the counter reaches **TIMER_CMP_*** value, the timer is not disabled; instead, the counter and irq is reset to 0. Since the timer remains enabled, the counter continues to increment on every positive edge of the clock until it again /reaches **TIMER_CMP_*** value. This process repeats cyclically.
- In 64-bit mode, the Compare Clear mode is enabled when the CMP_CLR_BIT bitfield in **CFG** CSR is set to 1. When both Timer_lo counter reaches **Timer_CMP_LO**, and, Timer_hi counter reaches **TIMER_CMP_HI**, the timer is not disabled; instead, the irq_lo and both counters are reset to 0. As both timers remain enabled, the counters continue to increment on each positive clock edge until Timer_lo reaches **TIMER_CMP_LO** and Timer_hi reaches **TIMER_CMP_HI** again. This process repeats cyclically.

4.2.3 Register Descriptions

Register 1: Timer Configuration Register (CFG)

Property	Value
Base address	SoC-defined
Byte offset	0x000
Reset value	0x00000000
Access type	RW

Register 4.4: TIMER CONFIGURATION REGISTER (CFG)



Functional Description.

The Control Register enables the counter and sets the prescaler. The full 32-bit word is stored, so the reserved bits are writable and read back their last written value, but they have no effect on operation.

Bit(s)	Name	Type	Reset	Description
31:6	reserved	RW	0	Reserved. Stored but have no effect.
5:3	PSC	RW	0	Prescaler. The counter increments once every (PSC + 1) clock cycles. PSC = 0 selects normal mode (increment every cycle).
2:1	reserved	RW	0	Reserved. Stored but have no effect.
0	EN	RW	0	Enable. When 1, the counter runs; when 0, the counter is frozen at its current value.

Initialization:

When the HRESETn signal is low, CSRs default to 0 and outputs are low.

At every positive edge of either the HCLK or reference clock the CSRs are updated based on APB signals.

Initializing the Prescaler:

By default prescalers are disabled, set PRESCALER_{ENBIT} of CFGR_{EG_L} or CFGR_{EG_H} ICSR to enable the prescaler.

If the Prescalers are enabled, Write to the PRESCALER_{COUNT} of CFGR_{EG_L} or CFGR_{EG_H} ICSR to specify the prescaler value.

Initializing the Timer counter:

Write to CSRs TIMER_{VAL_L} and TIMER_{VAL_H} to specify the initial counter value for Timer_i or Timer_h respectively.

Write to CSRs TIMER_{CMP_L} and TIMER_{CMP_H} to specify the compare count for Timer_i or Timer_h respectively.

Write '1' to either ONE_SHOT_{BIT} or CMPCLR_{BIT} of CFGR_{EG_L} or CFGR_{EG_H} ICSR to select the mode.

By default interrupts are disabled, set IRQ_{BIT} of CFGR_{EG_L} or CFGR_{EG_H} ICSR to enable the interrupts irq_i or irq_h.

By default input events are disabled, set IEM_{BIT} of CFGR_{EG_L} or CFGR_{EG_H} ICSR to enable the input event events_i or events_h.

By default reference clock is disabled, set REFCLK_{ENBIT} of CFGR_{EG_L} or CFGR_{EG_H} ICSR to enable the reference clock.

By default 64 bit mode is disabled and 32 bit mode is enabled, set MODE_{64BIT} of CFGR_{EG_L} OCSR to enable the 64 bit mode.

Set $\text{MODE}_{MTIME}_{BIT}$ of CFGR_{EG_L} or CFGR_{EG_H} ICSR to generate interrupt irq_i or irq_h for 64 bit timer respectively.

Start the timer:

Timer can be started in the following 3 ways:

When ENABLE_{BIT} of CFGR_{EG_L} or CFGR_{EG_H} ICSR is '1' for timer_i or timer_h respectively.

When the event_i or event_h signal is read as '1' and IEM_{BIT} of CFGR_{EG_L} or CFGR_{EG_H} ICSR is '1' for timer_i or timer_h respectively.

When TIMER_{START_L} or TIMER_{START_H} is any value other than '0' for timer_i or timer_h respectively.

Once the timer is started FW can observe the counter value getting incremented in the TIMER_{VAL_L} or TIMER_{VAL_H} ICSR.

Stop the timer:

Timer can be stopped in the following 2 ways:

When ENABLE_{BIT} of CFGR_{EG_L} or CFGR_{EG_H} ICSR is '0' for timer_i or timer_h respectively.

When stop timer_i is '1'.

Once the timer is stopped FW can observe the counter value remain the same in the TIMER_{VAL_L} or TIMER_{VAL_H} ICSR.

Reset the timer:

Timer can be reset in the following 2 ways:

When the $RESET_{BIT}$ in the CFG_{REG_L} or CFG_{REG_H} ICSR is '1' for timer_l or timer_h respectively.

When the $RESET_L$ in the $TIMER_{RESET_L}$ or $TIMER_{RESET_H}$ ICSR is '1' for timer_l or timer_h respectively.

Once the timer is stopped FW can observe the counter value reset to '0' in the $TIMER_{VAL_L}$ or $TIMER_{VAL_H}$ ICSR for timer_l or timer_h respectively.

Interrupt generation:

If the IRQ_{BIT} of CFG_{REG_L} is '1', irq_{lo} will be asserted when the counter value of Timer_l reaches the $TIMER_{CMP_L}$.

If the IRQ_{BIT} of CFG_{REG_H} is '1', irq_{ho} will be asserted when the counter value of Timer_h reaches the $TIMER_{CMP_H}$.

APB ADVANCED TIMER generates PWM for the external devices connected to $CORE_{VMCU}$ and the output events for the timers are called "channels". These four timers can be configured independently to support four unique PWM generations in parallel.

Multiple clock sources:

low_{speed} clock

HCLK clock

Multiple input sources:

PWM output signals of all 4 timers as input to the APB Advanced Timer.

GPIOs are inputs to the APB Advanced Timer

Configurable input trigger modes for each timer

Configurable prescaler for each timer

Configurable counting mode for each timer

Four comparators for each timer. Configurable channel comparator threshold and comparator operation for each comparator.

Four configurable output events

4-bit PWM output for each timer.

Configurable clock gating of each timer

Block Architecture

APB ADVANCED TIMER supports four 16-bit independent Timers. Each 16-bit timer has submodules, including an input stage, prescaler, up-down counter, and comparators. Each timer has its own set of CSRs that are used to configure various submodules.

APB timers can generate PWM signals in the following combination: -

A single timer can generate one 4-bit PWM.

Two timers can generate two 4-bit PWMs in parallel.

Three timers can generate three 4-bit PWMs in parallel.

Four timers can also generate four 4-bit PWMs in parallel.

APB ADVANCED TIMER also generates a 4-bit output event signal to the CPU subsystem, depending on the configuration of $REG_{EVENTCFGCSR}$.

The figure below is a high-level block diagram of the APB ADVANCED TIMER module:-

APB ADVANCED TIMER Block Diagram Timer Module

The timer module's main objective is to generate a PWM signal based on the external input/stimulus provided. The timer module has various submodules/components like the Timer Controller, Input stage, Prescaler, Updown counter, and Comparators. The timer controller manages all the other submodules through a few control signals like active, controller reset, and update. FW performs Initialization and drives various configuration CSR.

To generate the PWM, the data flows through the following submodule: - (ext_{sig}) → input stage →

prescaler → *updowncounter* → *comparators* → (*PWM*)

TIMER Block Diagram Timer Controller

The timer controller generates important signals, like active, update, and reset, arm and update counter. It controls other submodules through these signals.

The active signal: It is a control signal through which a sub-module can either enable or disable its operations.

The update signal: It informs the sub-module when to update the latest configured CSR values to perform their operations.

The reset signal: It resets the submodules.

The arm signal: It is passed to input stage for further processing of the input signal.

The update counter signal: It informs the updown counter to update the latest configured CSR values to perform its operations.

The active signal is enabled(1) or disabled(0) using below conditions:

The active signal for each timer is driven high ('1') when the corresponding START bitfield is set to '1' in its respective $REG_{TIMxCMDCSR}$ (where $x = 0$ to 3 for Timer0 to Timer3).

The active signal for each timer is driven low ('0') when the corresponding START bitfield is '0' and the STOP bitfield is set to '1' in its respective $REG_{TIMxCMDCSR}$ (where $x = 0$ to 3 for Timer0 to Timer3).

The update and update counter signals are always driven by the UPDATE bitfield value in the $REG_{TIMxCMDCSR}$ (where $x = 0$ to 3 for Timer0 to Timer3), and the controller reset signal is driven by the RESET bitfield value in the $REG_{TIMxCMDCSR}$ (where $x = 0$ to 3 for Timer0 to Timer3). The update, update counter and reset signals are passed to all sub-modules if any one of the following conditions are met:

If the START bitfield is '0' in the respective $REG_{TIMxCMDCSR}$ (where $x = 0$ to 3 for Timer0 to Timer3).

If the START bitfield is '1' in the respective $REG_{TIMxCMDCSR}$ and the active signal is driven high ('1'), this condition applies.

Input Stage

Input stage receives the 48-bit input (i.e., 32-bit ext_sig_i from APB GPIO and 16-bit PWM signals (4-bit PWM output signal of the 4 timer each)). Depending on CSR configurations, it selects the clock, input pin, and operating mode (where $x = 0$ to 3 for Timer0 to Timer3). If the INSEL bitfield is '7', then 7th bit of 48-bit input is selected for further processing. Input stage also receives the active signal from the timer controller (where $x = 0$ to 3 for Timer0 to Timer3):

If $CLKSEL = 1$, it processes the input signal as per MODE at every positive edge of the reference clock.

If $REF_CLK_ENBIT = 0$, it processes the input signal as per MODE at every positive edge of the system clock (HCLK).

When the timer controller asserts the active signal, then at every positive edge of the selected clock and selected input signal, the Input stage uses the bitfield MODE in $REG_{TIMxCFGCSR}$ (where $x = 0$ to 3 for Timer0 to Timer3) to generate an output event signal according to the information below:

If MODE is 3'b000

The output event is always high

If MODE is 3'b001

The output event is the negation of the signal selected

If MODE is 3'b010

The output event is the same as the input signal selected

If MODE is 3'b011

The output event is high at the rising edge of the selected signal in sync with the clock.

If MODE is 3'b100

The output event is high at the falling edge of the selected signal in sync with the clock.

If MODE is 3'b101

The output event is high at both the rising edge and falling edge of the selected signal in sync with the clock.

If MODE is 3'b110

If the timer is armed, i.e., the ARM is high, then the event is made high for the rising edge of the selected signal and remains the same until the next rising edge of the signal. If the ARM CSR is low, then the output event is low forever.

If MODE is 3'b111

If the timer is armed, i.e., the CSR ARM is high, then the event is made high for the falling edge of the selected signal and remains the same until the next falling edge of the signal. If the ARM CSR is low, then the output event is low forever.

Prescaler

The prescaler converts a high-frequency input event ($event_i$) into a low-frequency output event ($event_o$).

Prescaler maintains a prescaler counter whose initial value is '0'. For every positive edge of the HCLK, when the timer controller asserts the active signal and input-stage asserts the event ($event_i$), the prescaler counter is incremented by 1. When the counter reaches the value of $COUNT_{PRESCALER}$, it generates an output signal ($event_o$) for the Up-Down counter module. In the next positive edge of the clock, the counter is reset to 0.

The output signal ($event_o$) is de-asserted under the following conditions :

A system reset is received.

A reset is issued by the timer controller module.

The input-stage module de-asserts the input event signal (i.e., $event_i$ goes low).

The timer is disabled by the timer controller.

Updown counter

The updown counter manages the timer counter values based on CSR configurations and generates the following outputs:

The counter value.

The end event.

The output event.

The output event generated from the prescaler sub-module is provided as the input for the updown counter. At every positive edge of the clock, if the active signal is '1', then the output event is driven by the value of the output event generated from the prescaler. The active, controller reset, and update signals are provided by the Timer controller. The updown counter maintains a counter and direction (0- up and 1- down). During the initialization, the counter value is set to $COUNT_{START}$ and the direction to 0. At every positive edge of the clock, if

if the SAWTOOTH bitfield is '1':

The counter is incremented till it reaches the value of $COUNT_{END}$, then an end event is generated.

After the end event is generated, the counter is reset back to the value of the $COUNT_{START}$ bitfield, and this process is repeated.

if the SAWTOOTH bitfield is '0':

The counter is incremented till it reaches the value of $COUNT_{END}$.

Then the counter is decremented till it reaches the value of $COUNT_{START}$. (counter goes in a sawtooth fashion)

Now, an end event is generated. This process is repeated to generate multiple end events.

Re-initialization of the Updown counter can be done in any of the following scenarios.

The update counter signal is '1' and the controller is inactive (active signal is '0').

The update counter signal is '1' and the end event is generated.

Reset signal is '1'.

If the update counter signal is '1' and either controller is active or the end event is not generated, then the updown counter goes into pending initialization stage. In the pending initialization stage, the updown counter is re-initialized when the next end event is generated, irrespective of the update counter signal value at that instance of time.

At every positive edge of the clock, the counter value is updated in the $REG_{TIMx}COUNTERCSR$ (where $x = 0$ to 3 for Timer 0 to Timer 3) for the respective counter. If the hard reset is '0', then all the CSR and internal metadata are reset to their default values.

Each timer has 4 comparators that can act independently, and each comparator generates a 1-bit PWM output. Comparator compares the timer counter value with the compare value and, based on the CSR configurations of output mode, generates a PWM output. The counter value, end event, and the output event generated in the updown counter are provided as input to the comparator. The active, reset, and update signals are provided by the Timer controller.

At every positive edge of the clock, when the output event coming out of the updown counter is '1' and the active signal is '1', the comparator checks for the following two internal events that can happen,

(match_{event}) is set to '1' when the counter value reaches the $COMP_{THRESHOLD}$.

(event₂) set to '1' in the following two scenarios :

When the SAWTOOTH bitfield is '1' and the end event is '1'.

When SAWTOOTH is a bitfield of '0' and the counter value reaches the $COMP_{THRESHOLD}$.

The PWM output $ch_{xo}[y]$ is generated (where $x = 0$ to 3 for Timer 0 to Timer 3 and $y = 0, 1, 2$ and 3 refer to 0th, 1st, 2nd and 3rd comparators).

If $COMP_{OP}$ value is 3'b000 (OP_{SET})

If a match_{event} is high

The PWM output $ch_{xo}[y]$ is made high

Else, if a match_{event} is low

The PWM output $ch_{xo}[y]$ remains the same.

If $COMP_{OP}$ value is 3'b001 (OP_{TGRST})

When Sawtooth Mode is ON

If a match_{event} is high

The PWM output $ch_{xo}[y]$ is toggled.

Else, if event₂ is high

The PWM output $ch_{xo}[y]$ is made low.

When Sawtooth Mode is OFF

If match_{event} is high and event₂ is low

The PWM output $ch_{xo}[y]$ is toggled.

event₂ is made high.

Else, if match_{event} is high and event₂ is high

The PWM output $ch_{xo}[y]$ is made low

event₂ is made low.

If $COMP_{OP}$ value is 3'b010 (OP_{ETRST})

When Sawtooth Mode is ON

If a match_{event} is high

the PWM output $ch_{xo}[y]$ is made high

Else, if event_2 is high
then PWM output $\text{ch}_{xo}[y]$ is made low.
When Sawtooth Mode is OFF
If $\text{match}_{\text{event}_2}$ is high and event_2 is low
The PWM output $\text{ch}_{xo}[y]$ is made high
 event_2 is made high.
Else, if $\text{match}_{\text{event}_2}$ is high and event_2 also is high
PWM output $\text{ch}_{xo}[y]$ is made low
 event_2 is made low.
If COMP_O Pvalue is 3'b011 ($\text{OP}_{T\text{OG}}$)
If a $\text{match}_{\text{event}_2}$ is high
The PWM output $\text{ch}_{xo}[y]$ is toggled
Else, if a $\text{match}_{\text{event}_2}$ is low
The PWM output $\text{ch}_{xo}[y]$ remains the same.
If COMP_O Pvalue is 3'b100 ($\text{OP}_{R\text{ST}}$)
If a $\text{match}_{\text{event}_2}$ is high
The PWM output $\text{ch}_{xo}[y]$ is made low
Else, if a $\text{match}_{\text{event}_2}$ is low
The PWM output $\text{ch}_{xo}[y]$ remains the same.
If COMP_O Pvalue is 3'b101 ($\text{OP}_{T\text{OGSET}}$)
When Sawtooth Mode is ON
If a $\text{match}_{\text{event}_2}$ is high
The PWM output $\text{ch}_{xo}[y]$ is toggled
Else, if event_2 is high
then PWM output $\text{ch}_{xo}[y]$ is made high.
When Sawtooth Mode is OFF
If $\text{match}_{\text{event}_2}$ is high and event_2 is low
The PWM output $\text{ch}_{xo}[y]$ is toggled
 event_2 is made high
Else, if $\text{match}_{\text{event}_2}$ is high and event_2 also is high
The PWM output $\text{ch}_{xo}[y]$ is made high
 event_2 is made low
If COMP_O Pvalue is 3'b110 ($\text{OP}_{R\text{STSET}}$)
When Sawtooth Mode is ON
If a $\text{match}_{\text{event}_2}$ is high
The PWM output $\text{ch}_{xo}[y]$ is made low
Else, if event_2 is high
The PWM output $\text{ch}_{xo}[y]$ is made high

When Sawtooth Mode is OFF

If $\text{match}_{event1} \text{ is high and } event_2 \text{ is low}$

The PWM output $ch_{xo}[y]$ is made low

$event_2$ is made high

Else, if $\text{match}_{event1} \text{ is high and } event_2 \text{ also is high}$

The PWM output $ch_{xo}[y]$ is made high

$event_2$ is made low.

By default, the PWM output remains the same (state remains the same until further change in input), and $event_2$ is kept low. The PWM output is set to 0. When either the hardware reset is triggered or the controller reset is '1'. $COMP_THRESHOLD$

Based on the detailed working of the submodules above, for each Timer module, at every positive edge of the selected clock and when the timer is active, the following operation is performed:

Input stage consumes 48 bits (i.e., 32-bit ext_sig and 16-bit PWM output signal for all the 4 timers) and processes it accordingly.

The Event signal generated in the input stage is scaled down to the output scaled event based on the prescaler value by the prescaler sub-module.

The above output scaled events generated go to the updown counter.

Depending on various FW configurations of SAWTOOTH, $COUNT_START$, and $COUNT_END$. The counter value, end event

In each of the comparators, the counter value is compared against the $COMP_THRESHOLD$, and a 1-bit PWM is generated based on $COMP_OP$.

4 comparator submodules generate 4 4-bit PWM signals

The above process is repeated until the timer is disabled.

APB ADVANCED TIMER has 4 timer modules, which can generate 4 independent 4-bit PWMs. Working of APB ADVANCED TIMER for output event generation:

Apart from generating PWM signals, the APB ADVANCED TIMER can also generate output events using bitfields in the REG_EVENT_CFGCSR .

For every positive edge of the HCLK, the output events are generated after the following operation is performed.

The $OUT_SEL_VT_ENABLE$ bitfield determines which output event lines ($events_o[0]$ to $events_o[3]$) are enabled.

Each bitfield OUT_SEL_VT0 to OUT_SEL_VT3 selects one of the 16 PWM signals ($ch_{0o}[0]$ to $ch_{0o}[3]$, $ch_{1o}[0]$ to $ch_{1o}[3]$, $ch_{2o}[0]$ to $ch_{2o}[3]$, $ch_{3o}[0]$ to $ch_{3o}[3]$).

The output events are triggered at the rising edge of the selected PWM signal and will be de-asserted in the next HCLK clock cycle.

For every positive edge of the HCLK, the same process is repeated to generate the output events.

Example:

If bit 0 of $OUT_SEL_VT_ENABLE$ is set ($OUT_SEL_VT_ENABLE[0] = 1$) and OUT_SEL_VT0 is set to 4, then $events_o[0]$ will assert.

The figure below depicts the connections between the APB ADVANCED TIMER and the rest of the modules in CORE-V-MCU:-

APB ADVANCED TIMER CORE-V-MCU connections diagram

The ext_sig_i input to the APB ADVANCED TIMER is provided by the APB GPIO.

APB ADVANCED TIMER processes these input signals based on the various CSR configurations.

APB ADVANCED TIMER generates a few output event signals that are further passed as interrupts to the Core complex.

APB ADVANCED TIMER generates PWM outputs, which are passed to the external devices through the

I/O mux.

Programmers View:

APB ADVANCED TIMER has 4 Timers, and the following programming model is followed: Initial Configurations:

There are CSR bitfields in the APB advanced timer that are required to be configured before any operations are initiated. Timer module specific configurations:

As we have 4 Timer modules. Each timer has to be configured with appropriate values.

Configure input clock source using CLKSEL bitfield in the $REG_{TIMxCFGCSR}$ (where $x = 0$ to 3 for Timer0 to Timer3).

Configure input trigger mode using the MODE bitfield in the $REG_{TIMxCFGCSR}$ (where $x = 0$ to 3 for Timer0 to Timer3).

Configure which input has to be selected using the INSEL bitfield in the $REG_{TIMxCFGCSR}$ (where $x = 0$ to 3 for Timer0 to Timer3).

Configure prescaler value for scaling down the frequency using the PRESC bitfield in the $REG_{TIMxCFGCSR}$ (where $x = 0$ to 3 for Timer0 to Timer3).

Configure sawtooth mode through which the updown down counter operates using the SAWTOOTH bitfield in the $REG_{TIMxCFGCSR}$ (where $x = 0$ to 3 for Timer0 to Timer3).

Configure updown counter start value and end value using $COUNT_{START}$ and $COUNT_{END}$ bit field respectively in the REG_{TIMxCR} (where $x = 0$ to 3 for Timer0 to Timer3).

Configure comparator 0 operation and comparator 0 threshold using $COMP_{OP}$ and $COMP_{THRESHOLD}$ bit field respectively in the $REG_{TIMxCCR0}$ (where $x = 0$ to 3 for Timer0 to Timer3).

Configure comparator 1 operation and comparator 1 threshold using $COMP_{OP}$ and $COMP_{THRESHOLD}$ bit field respectively in the $REG_{TIMxCCR1}$ (where $x = 0$ to 3 for Timer0 to Timer3).

Configure comparator 2 operation and comparator 2 threshold using $COMP_{OP}$ and $COMP_{THRESHOLD}$ bit field respectively in the $REG_{TIMxCCR2}$ (where $x = 0$ to 3 for Timer0 to Timer3).

Configure comparator 3 operation and comparator 3 threshold using $COMP_{OP}$ and $COMP_{THRESHOLD}$ bit field respectively in the $REG_{TIMxCCR3}$ (where $x = 0$ to 3 for Timer0 to Timer3).

Common configurations:

These configurations are common for 4 TIMERS. Typically, these are used to enable or disable output events, clock for TIMERS, and select the output events from a group of 16 PWM events.

Configure output select event enable that controls to enable or disable any of the 4-bit output events, using $OUT_{SEL_{EVTEN}}$.

Configure output event 0, select the value that is used to select an event from the 16-bit PWM output using the $OUT_{SEL_{EVT0}}$ bit field in the $REG_{EVENTCFG}$.

Configure output event 1, select the value that is used to select an event from the 16-bit PWM output using the $OUT_{SEL_{EVT1}}$ bit field in the $REG_{EVENTCFG}$.

Configure output event 2 to select a value that is used to select an event from the 16-bit PWM output using the $OUT_{SEL_{EVT2}}$ bit field in the $REG_{EVENTCFG}$.

Configure output event 3, select the value that is used to select an event from the 16-bit PWM output using the $OUT_{SEL_{EVT3}}$ bit field in the $REG_{EVENTCFG}$.

Enable or disable clocks for each TIMER using the CLK_{ENABLE} bit field in the REG_{CHEN} .

Control configurations/operations:

There are CSR bitfields in the APB advanced timer that control operations of each of the timer modules and their submodules.

Set the START bitfield in the $REG_{TIMxMDCSR}$ (where $x = 0$ to 3 for Timer0 to Timer3) to start the Timer and its submodule.

Set the STOP bitfield in the $REG_{TIMxMDCSR}$ (where $x = 0$ to 3 for Timer0 to Timer3) to stop/halt/pause the Timer and its submodule.

Set the UPDATE bitfield in the $REG_{TIMx}MDCSR$ (where $x = 0$ to 3 for Timer0 to Timer3) to Re-Initialization with the

Set the RESET bitfield in the $REG_{TIMx}MDCSR$ (where $x = 0$ to 3 for Timer0 to Timer3) to reset the timer and its submode

set the ARM bitfield in the $REG_{TIMx}MDCSR$ (where $x = 0$ to 3 for Timer0 to Timer3) to modify the inputs in the input stage

Status configurations:

The counter values of all 4 Timers can be read via the following CSR bitfields in the APB advanced timer.

Use the $TX_COUNTER$ bitfields in the respective $REG_{TIMx}COUNTERCSR$ (where $x = 0$ to 3 for Timer0 to Timer3) to read

APB ADVANCED TIMER CSRs

Refer to Memory Map Map for the peripheral domain addresses of the APB ADVANCED TIMER. NOTE: APB ADVANCED TIMER CSRs are non-volatile, meaning that their read value will not be changed by the hardware. A CSR's volatility is indicated by its "type". Details of CSR access type are explained here.

$REG_{TIM0}MD$

Address Offset=0x000

Type: non-volatile

Field

Bits

Default Value

Access

Description

RESERVED

31:5

0

—

Reserved

ARM

4:4

0

RW

set this bitfield to modify the input data as per description in input stage when MODE bitfield value is 6 or 7

RESET

3:3

0

RW

set this bitfield to reset the timer, even when the timer is active for PWM generation.

UPDATE

2:2

0

RW

set this bitfield to update or re-initialize the timer when the timer is stopped

STOP

1:1

0

RW

set this bitfield to stop the timer and its sub modules operations

START

0:0

0

RW

set this bitfield to Start the timer operation to generate PWM output $REG_{TIM0CFG}$

Address Offset=0x004

Type: non-volatile

Field

Bits

Default Value

Access

Description

RESERVED

31:24

0

—

Reserved

PRESC

23:16

0

RW

prescaler compare value configuration bitfield

RESERVED

15:13

0

—

Reserved

SAWTOOTH

12:12

1

RW

center-aligned mode configuration bitfield

1'b0: The counter counts up and down alternatively

1'b1: The counter counts up and resets to 0 when it reaches the threshold.

CLKSEL

11:11

0

RW

clock source configuration bitfield

1'b0: HCLK

1'b1: Reference clock at 32kHz

MODE

10:8

0

RW

Input stage triggers an output event as per the mode configuration bitfield

3'h0: Trigger output event at each clock cycle

3'h1: Trigger output event if input source is 0

3'h2: Trigger output event if input source is 1

3'h3: Trigger output event on input source rising edge

3'h4: Trigger output event on input source falling edge

3'h5: Trigger output event on input source falling or rising edge

3'h6: Trigger output event on input source rising edge when armed

3'h7: Trigger output event on input source falling edge when armed

INSEL

7:0

0

RW

input source configuration bitfield

0-31: GPIO[0] to GPIO[31]

32 - $ch_{0o}[0]$ *isselected*

33 - $ch_{0o}[1]$ *isselected*

34 - $ch_{0o}[2]$ *isselected*

35 - $ch_{0o}[3]$ *isselected*

36 - $ch_{1o}[0]$ *isselected*

37 - $ch_{1o}[1]$ *isselected*

38 - $ch_{1o}[2]$ *isselected*

39 - $ch_{1o}[3]$ *isselected*

40 - $ch_{2o}[0]$ *isselected*

41 - $ch_{2o}[1]$ *isselected*

42 - $ch_{2o}[2]$ *isselected*

43 - $\text{ch}_{2o}[3]_{\text{isselected}}$

44 - $\text{ch}_{3o}[0]_{\text{isselected}}$

45 - $\text{ch}_{3o}[1]_{\text{isselected}}$

46 - $\text{ch}_{3o}[2]_{\text{isselected}}$

47 - $\text{ch}_{3o}[3]_{\text{isselected}} \text{REG}_{TIM0TH}$

Address Offset=0x008

Type: non-volatile

Field

Bits

Default Value

Access

Description

COUNT_{END}

31:16

0

RW

End value for the updown counter

COUNT_{START}

15:0

0

RW

Start value for the updown counter $\text{REG}_{TIM0CH0TH}$

Address Offset=0x00C

Type: non-volatile

Field

Bits

Default Value

Access

Description

RESERVED

31:19

0

—

Reserved

COMP_{OP}

18:16

0

RW

It decides the comparator operation to be performed to drive the output PWM ch_{0o}[0].*Detailed description is provided in com*

COMP_THRESHOLD

15:0

0

RW

Timer0 Channel 0 comparator threshold value (Timer0 updown counter value is compared with the COMP_THRESHOLD

Address Offset=0x010

Type: non-volatile

Field

Bits

Default Value

Access

Description

RESERVED

31:19

0

—

Reserved

COMP_{OP}

18:16

0

RW

It decides the comparator operation to be performed to drive the output PWM ch_{0o}[1].*Detailed description is provided in com*

COMP_THRESHOLD

15:0

0

RW

Timer0 Channel 1 comparator threshold value (Timer0 updown counter value is compared with the COMP_THRESHOLD

Address Offset=0x014

Type: non-volatile

Field

Bits

Default Value

Access

Description

RESERVED

31:19

0

—

Reserved

COMP_{OP}

18:16

0

RW

It decides the comparator operation to be performed to drive the output PWM ch_{0o}[2].*Detailed description is provided in con*COMP_{THRESHOLD}

15:0

0

RW

Timer0 Channel 2 comparator threshold value (Timer0 updown counter value is compared with the COMP_{THRESHOLD}

Address Offset=0x018

Type: non-volatile

Field

Bits

Default Value

Access

Description

RESERVED

31:19

0

—

Reserved

COMP_{OP}

18:16

0

RW

It decides the comparator operation to be performed to drive the output PWM ch_{0o}[3].*Detailed description is provided in con*COMP_{THRESHOLD}

15:0

0

RW

Timer0 Channel 3 comparator threshold value (Timer0 updown counter value is compared with the COMP_{THRESHOLD}

Address Offset=0x01C

Type: non-volatile

Field

Bits

Default Value

Access

Description

RESERVED

31:18

0

—

Reserved

FLT

23:16

0

RW

FLT (Not used in the current implementation)

LUT

15:0

0

RW

LUT (Not used in the current implementation) $REG_{TIM0CH1LUT}$

Address Offset=0x020

Type: non-volatile

Field

Bits

Default Value

Access

Description

RESERVED

31:18

0

—

Reserved

FLT

23:16

0

RW

FLT (Not used in the current implementation)

LUT

15:0

0

RW

LUT (Not used in the current implementation) $\text{REG}_{TIM0_C H2_LUT}$

Address Offset=0x024

Type: non-volatile

Field

Bits

Default Value

Access

Description

RESERVED

31:18

0

—

Reserved

FLT

23:16

0

RW

FLT (Not used in the current implementation)

LUT

15:0

0

RW

LUT (Not used in the current implementation) $\text{REG}_{TIM0_C H3_LUT}$

Address Offset=0x028

Type: non-volatile

Field

Bits

Default Value

Access

Description

RESERVED

31:18

0

—

Reserved

FLT

23:16

0

RW

FLT (Not used in the current implementation)

LUT

15:0

0

RW

LUT (Not used in the current implementation) $REG_{TIM0_COUNTER}$

Address Offset=0x02C

Type: non-volatile

Field

Bits

Default Value

Access

Description

 $T0_COUNTER$

31:0

0

R

TIMER0 counter REG_{TIM1_CMD}

Address Offset=0x040

Type: non-volatile

Field

Bits

Default Value

Access

Description

RESERVED

31:5

0

—

Reserved

ARM

4:4

0

RW

set this bitfield to modify the input data as per description in input stage when MODE bitfield value is 6 or 7

RESET

3:3

0

RW

set this bitfield to reset the timer, even when the timer is active for PWM generation.

UPDATE

2:2

0

RW

set this bitfield to update or re-initialize the timer when the timer is stopped

STOP

1:1

0

RW

set this bitfield to stop the timer and its sub modules operations

START

0:0

0

RW

set this bitfield to Start the timer operation to generate PWM output $REG_{TIM1CFG}$

Address Offset=0x044

Type: non-volatile

Field

Bits

Default Value

Access

Description

RESERVED

31:24

0

—

Reserved

PRESC

23:16

0

RW

prescaler value configuration bitfield

RESERVED

15:13

0

—

Reserved

SAWTOOTH

12:12

1

RW

center-aligned mode configuration bitfield

1'b0: The counter counts up and down alternatively

1'b1: The counter counts up and resets to 0 when it reaches the threshold.

CLKSEL

11:11

0

RW

clock source configuration bitfield

1'b0: HCLK

1'b1: Reference clock at 32kHz

MODE

10:8

0

RW

Input stage triggers an output event as per the mode configuration bitfield

3'h0: Trigger output event at each clock cycle

3'h1: Trigger output event if input source is 0

3'h2: Trigger output event if input source is 1

3'h3: Trigger output event on input source rising edge

3'h4: Trigger output event on input source falling edge

3'h5: Trigger output event on input source falling or rising edge

3'h6: Trigger output event on input source rising edge when armed

3'h7: Trigger output event on input source falling edge when armed

INSEL

7:0

0

RW

input source configuration bitfield

0-31: GPIO[0] to GPIO[31]

32 - $\text{ch}_{0o}[0]$ *isselected*

33 - $\text{ch}_{0o}[1]$ *isselected*

34 - $\text{ch}_{0o}[2]$ *isselected*

35 - $\text{ch}_{0o}[3]$ *isselected*

36 - $\text{ch}_{1o}[0]$ *isselected*

37 - $\text{ch}_{1o}[1]$ *isselected*

38 - $\text{ch}_{1o}[2]$ *isselected*

39 - $\text{ch}_{1o}[3]$ *isselected*

40 - $\text{ch}_{2o}[0]$ *isselected*

41 - $\text{ch}_{2o}[1]$ *isselected*

42 - $\text{ch}_{2o}[2]$ *isselected*

43 - $\text{ch}_{2o}[3]$ *isselected*

44 - $\text{ch}_{3o}[0]$ *isselected*

45 - $\text{ch}_{3o}[1]$ *isselected*

46 - $\text{ch}_{3o}[2]$ *isselected*

47 - $\text{ch}_{3o}[3]$ *isselected* REG_{TIM1TH}

Address Offset=0x048

Type: non-volatile

Field

Bits

Default Value

Access

Description

COUNT_{END}

31:16

0

RW

End value for the updown counter

COUNT_{START}

15:0

0

RW

Start value for the updown counter $\text{REG}_{TIM1CH0TH}$

Address Offset=0x04C

Type: non-volatile

Field

Bits

Default Value

Access

Description

RESERVED

31:19

0

—

Reserved

COMP_{OP}

18:16

0

RW

It decides the comparator operation to be performed to drive the output PWM ch_{1o}[0]. *Detailed description is provided in the*

COMP_{THRESHOLD}

15:0

0

RW

Timer1 Channel 0 comparator threshold value (Timer1 updown counter value is compared with the COMP_{THRESHOLD})

Address Offset=0x050

Type: non-volatile

Field

Bits

Default Value

Access

Description

RESERVED

31:19

0

—

Reserved

COMP_{OP}

18:16

0

RW

It decides the comparator operation to be performed to drive the output PWM ch_{1o}[1]. *Detailed description is provided in the*

COMP_{THRESHOLD}

15:0

0

RW

Timer1 Channel 1 comparator threshold value (Timer1 updown counter value is compared with the COMP_{THRESHOLD})

Address Offset=0x054

Type: non-volatile

Field

Bits

Default Value

Access

Description

RESERVED

31:19

0

—

Reserved

COMP_{OP}

18:16

0

RW

It decides the comparator operation to be performed to drive the output PWM ch_{1o}[2].*Detailed description is provided in con*

COMP_{THRESHOLD}

15:0

0

RW

Timer1 Channel 2 comparator threshold value (Timer1 updown counter value is compared with the COMP_{THRESHOLD}

Address Offset=0x058

Type: non-volatile

Field

Bits

Default Value

Access

Description

RESERVED

31:19

0

—

Reserved

COMP_{OP}

18:16

0

RW

It decides the comparator operation to be performed to drive the output PWM $ch_{1o}[3]$. *Detailed description is provided in component*

$COMP_{THRESHOLD}$

15:0

0

RW

Timer1 Channel 3 comparator threshold value (Timer1 updown counter value is compared with the $COMP_{THRESHOLD}$)

Address Offset=0x05C

Type: non-volatile

Field

Bits

Default Value

Access

Description

RESERVED

31:18

0

—

Reserved

FLT

23:16

0

RW

FLT (Not used in the current implementation)

LUT

15:0

0

RW

LUT (Not used in the current implementation) $REG_{TIM1CH1LUT}$

Address Offset=0x060

Type: non-volatile

Field

Bits

Default Value

Access

Description

RESERVED

31:18

0

—

Reserved

FLT

23:16

0

RW

FLT (Not used in the current implementation)

LUT

15:0

0

RW

LUT (Not used in the current implementation) $REG_{TIM1CH2LUT}$

Address Offset=0x064

Type: non-volatile

Field

Bits

Default Value

Access

Description

RESERVED

31:18

0

—

Reserved

FLT

23:16

0

RW

FLT (Not used in the current implementation)

LUT

15:0

0

RW

LUT (Not used in the current implementation) $REG_{TIM1CH3LUT}$

Address Offset=0x068

Type: non-volatile

Field

Bits

Default Value

Access

Description

RESERVED

31:18

0

—

Reserved

FLT

23:16

0

RW

FLT (Not used in the current implementation)

LUT

15:0

0

RW

LUT (Not used in the current implementation) $REG_{TIM1_COUNTER}$

Address Offset=0x06C

Type: non-volatile

Field

Bits

Default Value

Access

Description

$T1_COUNTER$

31:0

0

R

TIMER1 counter REG_{TIM2_CMD}

Address Offset=0x080

Type: non-volatile

Field

Bits

Default Value

Access

Description

RESERVED

31:5

0

—

Reserved

ARM

4:4

0

RW

set this bitfield to modify the input data as per description in input stage when MODE bitfield value is 6 or 7

RESET

3:3

0

RW

set this bitfield to reset the timer, even when the timer is active for PWM generation.

UPDATE

2:2

0

RW

set this bitfield to update or re-initialize the timer when the timer is stopped

STOP

1:1

0

RW

set this bitfield to stop the timer and its sub modules operations

START

0:0

0

RW

set this bitfield to Start the timer operation to generate PWM output $REG_{TIM2CFG}$

Address Offset=0x084

Type: non-volatile

Field

Bits

Default Value

Access

Description

RESERVED

31:24

0

—

Reserved

PRESC

23:16

0

RW

prescaler value configuration bitfield

RESERVED

15:13

0

—

Reserved

SAWTOOTH

12:12

1

RW

center-aligned mode configuration bitfield

1'b0: The counter counts up and down alternatively

1'b1: The counter counts up and resets to 0 when it reaches the threshold.

CLKSEL

11:11

0

RW

clock source configuration bitfield

1'b0: HCLK

1'b1: Reference clock at 32kHz

MODE

10:8

0

RW

Input stage triggers an output event as per the mode configuration bitfield

3'h0: Trigger output event at each clock cycle

3'h1: Trigger output event if input source is 0

3'h2: Trigger output event if input source is 1

3'h3: Trigger output event on input source rising edge

3'h4: Trigger output event on input source falling edge

3'h5: Trigger output event on input source falling or rising edge

3'h6: Trigger output event on input source rising edge when armed

3'h7: Trigger output event on input source falling edge when armed

INSEL

7:0

0

RW

input source configuration bitfield

0-31: GPIO[0] to GPIO[31]

32 - $ch_{0o}[0]$ *isselected*

33 - $ch_{0o}[1]$ *isselected*

34 - $ch_{0o}[2]$ *isselected*

35 - $ch_{0o}[3]$ *isselected*

36 - $ch_{1o}[0]$ *isselected*

37 - $ch_{1o}[1]$ *isselected*

38 - $ch_{1o}[2]$ *isselected*

39 - $ch_{1o}[3]$ *isselected*

40 - $ch_{2o}[0]$ *isselected*

41 - $ch_{2o}[1]$ *isselected*

42 - $ch_{2o}[2]$ *isselected*

43 - $ch_{2o}[3]$ *isselected*

44 - $ch_{3o}[0]$ *isselected*

45 - $ch_{3o}[1]$ *isselected*

46 - $ch_{3o}[2]$ *isselected*

47 - $ch_{3o}[3]$ *isselected* REG_{TIM2TH}

Address Offset=0x088

Type: non-volatile

Field

Bits

Default Value

Access

Description

$COUNT_{END}$

31:16

0

RW

End value for the updown counter

$COUNT_{START}$

15:0

0

RW

Start value for the updown counter $REG_{TIM2CH0TH}$

Address Offset=0x08C

Type: non-volatile

Field

Bits

Default Value

Access

Description

RESERVED

31:19

0

—

Reserved

 $COMP_{OP}$

18:16

0

RW

It decides the comparator operation to be performed to drive the output PWM $ch_{2o}[0]$. *Detailed description is provided in com* $COMP_{THRESHOLD}$

15:0

0

RW

Timer2 Channel 0 comparator threshold value (Timer2 updown counter value is compared with the $COMP_{THRESHOLD}$)

Address Offset=0x090

Type: non-volatile

Field

Bits

Default Value

Access

Description

RESERVED

31:19

0

—

Reserved

$COMP_{OP}$

18:16

0

RW

It decides the comparator operation to be performed to drive the output PWM ch_{2o}[1].*Detailed description is provided in com*

 $COMP_{THRESHOLD}$

15:0

0

RW

Timer2 Channel 1 comparator threshold value (Timer2 updown counter value is compared with the $COMP_{THRESHOLD}$)

Address Offset=0x094

Type: non-volatile

Field

Bits

Default Value

Access

Description

RESERVED

31:19

0

—

Reserved

 $COMP_{OP}$

18:16

0

RW

It decides the comparator operation to be performed to drive the output PWM ch_{2o}[2].*Detailed description is provided in com*

 $COMP_{THRESHOLD}$

15:0

0

RW

Timer2 Channel 2 comparator threshold value (Timer2 updown counter value is compared with the $COMP_{THRESHOLD}$)

Address Offset=0x098

Type: non-volatile

Field

Bits

Default Value

Access

Description

RESERVED

31:19

0

—

Reserved

COMP_{OP}

18:16

0

RW

It decides the comparator operation to be performed to drive the output PWM ch_{2o}[3]. *Detailed description is provided in com*COMP_{THRESHOLD}

15:0

0

RW

Timer2 Channel 3 comparator threshold value (Timer2 updown counter value is compared with the COMP_{THRESHOLD}

Address Offset=0x09C

Type: non-volatile

Field

Bits

Default Value

Access

Description

RESERVED

31:18

0

—

Reserved

FLT

23:16

0

RW

FLT (Not used in the current implementation)

LUT

15:0

0

RW

LUT (Not used in the current implementation) REG_{IM2CH1LUT}

Address Offset=0x0A0

Type: non-volatile

Field

Bits

Default Value

Access

Description

RESERVED

31:18

0

—

Reserved

FLT

23:16

0

RW

FLT (Not used in the current implementation)

LUT

15:0

0

RW

LUT (Not used in the current implementation) $REG_{TIM2CH2LUT}$

Address Offset=0x0A4

Type: non-volatile

Field

Bits

Default Value

Access

Description

RESERVED

31:18

0

—

Reserved

FLT

23:16

0

RW

FLT (Not used in the current implementation)

LUT

15:0

0

RW

LUT (Not used in the current implementation) $REG_{TIM2CH3LUT}$

Address Offset=0x0A8

Type: non-volatile

Field

Bits

Default Value

Access

Description

RESERVED

31:18

0

—

Reserved

FLT

23:16

0

RW

FLT (Not used in the current implementation)

LUT

15:0

0

RW

LUT (Not used in the current implementation) $REG_{TIM2COUNTER}$

Address Offset=0x0AC

Type: non-volatile

Field

Bits

Default Value

Access

Description

$T2COUNTER$

31:0

0

R

TIMER2 counter $REG_{TIM3CMD}$

Address Offset=0x0C0

Type: non-volatile

Field

Bits

Default Value

Access

Description

RESERVED

31:5

0

—

Reserved

ARM

4:4

0

RW

set this bitfield to modify the input data as per description in input stage when MODE bitfield value is 6 or 7

RESET

3:3

0

RW

set this bitfield to reset the timer, even when the timer is active for PWM generation.

UPDATE

2:2

0

RW

set this bitfield to update or re-initialize the timer when the timer is stopped

STOP

1:1

0

RW

set this bitfield to stop the timer and its sub modules operations

START

0:0

0

RW

set this bitfield to Start the timer operation to generate PWM output $REG_{TIM3CFG}$

Address Offset=0x0C4

Type: non-volatile

Field

Bits

Default Value

Access

Description

RESERVED

31:24

0

—

Reserved

PRESC

23:16

0

RW

prescaler value configuration bitfield

RESERVED

15:13

0

—

Reserved

SAWTOOTH

12:12

1

RW

center-aligned mode configuration bitfield

1'b0: The counter counts up and down alternatively

1'b1: The counter counts up and resets to 0 when it reaches the threshold.

CLKSEL

11:11

0

RW

clock source configuration bitfield

1'b0: HCLK

1'b1: Reference clock at 32kHz

MODE

10:8

0

RW

Input stage triggers an output event as per the mode configuration bitfield

3'h0: Trigger output event at each clock cycle

3'h1: Trigger output event if input source is 0

3'h2: Trigger output event if input source is 1

3'h3: Trigger output event on input source rising edge

3'h4: Trigger output event on input source falling edge

3'h5: Trigger output event on input source falling or rising edge

3'h6: Trigger output event on input source rising edge when armed

3'h7: Trigger output event on input source falling edge when armed

INSEL

7:0

0

RW

input source configuration bitfield

0-31: GPIO[0] to GPIO[31]

32 - $ch_{0o}[0]$ *isselected*

33 - $ch_{0o}[1]$ *isselected*

34 - $ch_{0o}[2]$ *isselected*

35 - $ch_{0o}[3]$ *isselected*

36 - $ch_{1o}[0]$ *isselected*

37 - $ch_{1o}[1]$ *isselected*

38 - $ch_{1o}[2]$ *isselected*

39 - $ch_{1o}[3]$ *isselected*

40 - $ch_{2o}[0]$ *isselected*

41 - $ch_{2o}[1]$ *isselected*

42 - $ch_{2o}[2]$ *isselected*

43 - $ch_{2o}[3]$ *isselected*

44 - $ch_{3o}[0]$ *isselected*

45 - $ch_{3o}[1]$ *isselected*

46 - $ch_{3o}[2]$ *isselected*

47 - $ch_{3o}[3]$ *isselected* REG_{TIM3TH}

Address Offset=0x0C8

Type: non-volatile

Field

Bits

Default Value

Access

Description

COUNT_{END}

31:16

0

RW

End value for the updown counter

COUNT_{START}

15:0

0

RW

Start value for the updown counter REG_{TIM3CH0TH}

Address Offset=0x0CC

Type: non-volatile

Field

Bits

Default Value

Access

Description

RESERVED

31:19

0

—

Reserved

COMP_{OP}

18:16

0

RW

It decides the comparator operation to be performed to drive the output PWM ch_{3o}[0].*Detailed description is provided in com*

COMP_{THRESHOLD}

15:0

0

RW

Timer3 Channel 0 comparator threshold value (Timer3 updown counter value is compared with the COMP_{THRESHOLD}

Address Offset=0x0D0

Type: non-volatile

Field

Bits

Default Value

Access

Description

RESERVED

31:19

0

—

Reserved

 $COMP_{OP}$

18:16

0

RW

It decides the comparator operation to be performed to drive the output PWM $ch_{3o}[1]$. *Detailed description is provided in com*

 $COMP_{THRESHOLD}$

15:0

0

RW

Timer3 Channel 1 comparator threshold value (Timer3 updown counter value is compared with the $COMP_{THRESHOLD}$

Address Offset=0x0D4

Type: non-volatile

Field

Bits

Default Value

Access

Description

RESERVED

31:19

0

—

Reserved

 $COMP_{OP}$

18:16

0

RW

It decides the comparator operation to be performed to drive the output PWM $ch_{3o}[2]$. *Detailed description is provided in com*

 $COMP_{THRESHOLD}$

15:0

0

RW

Timer3 Channel 2 comparator threshold value (Timer3 updown counter value is compared with the $COMP_{THRESHOLD}$)

Address Offset=0x0D8

Type: non-volatile

Field

Bits

Default Value

Access

Description

RESERVED

31:19

0

—

Reserved

$COMP_{OP}$

18:16

0

RW

It decides the comparator operation to be performed to drive the output PWM $ch_{3o}[3]$. *Detailed description is provided in comparison mode.*

$COMP_{THRESHOLD}$

15:0

0

RW

Timer3 Channel 3 comparator threshold value (Timer3 updown counter value is compared with the $COMP_{THRESHOLD}$)

Address Offset=0x0DC

Type: non-volatile

Field

Bits

Default Value

Access

Description

RESERVED

31:18

0

—

Reserved

FLT

23:16

0

RW

FLT (Not used in the current implementation)

LUT

15:0

0

RW

LUT (Not used in the current implementation) $REG_{TIM3CH1LUT}$

Address Offset=0x0E0

Type: non-volatile

Field

Bits

Default Value

Access

Description

RESERVED

31:18

0

—

Reserved

FLT

23:16

0

RW

FLT (Not used in the current implementation)

LUT

15:0

0

RW

LUT (Not used in the current implementation) $REG_{TIM3CH2LUT}$

Address Offset=0x0E4

Type: non-volatile

Field

Bits

Default Value

Access

Description

RESERVED

31:18

0

—

Reserved

FLT

23:16

0

RW

FLT (Not used in the current implementation)

LUT

15:0

0

RW

LUT (Not used in the current implementation) $REG_{TIM3CH3LUT}$

Address Offset=0x0E8

Type: non-volatile

Field

Bits

Default Value

Access

Description

RESERVED

31:18

0

—

Reserved

FLT

23:16

0

RW

FLT (Not used in the current implementation)

LUT

15:0

0

RW

LUT (Not used in the current implementation) $REG_{TIM3COUNTER}$

Address Offset=0x0EC

Type: non-volatile

Field

Bits

Default Value

Access

Description

$T3_{COUNTER}$

31:0

0

R

TIMER3 counter $REG_{EVENTCFG}$

Address Offset=0x100

Type: non-volatile

Field

Bits

Default Value

Access

Description

RESERVED

31:20

0

—

Reserved

$OUT_{SEL_{EVENTENABLE}}$

19:16

0

RW

Output event select ENABLE. Each bit represents an event enable for 4 bit $events_ooutput$.

0b'0000 - all the events are disabled

0b'0001 - $events_o[0]$ is enabled

0b'0010 - $events_o[1]$ is enabled

0b'0011 - $events_o[0]$ and $events_o[1]$ are enabled

0b'0100 - $events_o[2]$ is enabled

0b'0101 - $events_o[0]$ and $events_o[2]$ are enabled

0b'0110 - $events_o[1]$ and $events_o[2]$ are enabled

0b'0111 - $events_o[0]$, $events_o[1]$ and $events_o[2]$ are enabled

0b'1000 - $events_o[3]$ is enabled

0b'1001 - $events_o[0]$ and $events_o[3]$ are enabled

0b'1010 - $events_o[1]$ and $events_o[3]$ are enabled

0b'1011 - $events_o[0]$, $events_o[1]$ and $events_o[3]$ are enabled

0b'1100 - $events_o[2]$ and $events_o[3]$ are enabled

0b'1101 - $events_o[0]$, $events_o[2]$ and $events_o[3]$ are enabled

0b'1110 - $events_o[1]$, $events_o[2]$ and $events_o[3]$ are enabled

0b'1111 - all the events are enabled

OUT_{SELEVT3}

15:12

0

RW

OUT_{SELEVT3} select a signal that drives $events_o[3]$ from a group of 16 PWM outputs

0b'0000 - $ch_{0o}[0]$ is selected

0b'0001 - $ch_{0o}[1]$ is selected

0b'0010 - $ch_{0o}[2]$ is selected

0b'0011 - $ch_{0o}[3]$ is selected

0b'0100 - $ch_{1o}[0]$ is selected

0b'0101 - $ch_{1o}[1]$ is selected

0b'0110 - $ch_{1o}[2]$ is selected

0b'0111 - $ch_{1o}[3]$ is selected

0b'1000 - $ch_{2o}[0]$ is selected

0b'1001 - $ch_{2o}[1]$ is selected

0b'1010 - $ch_{2o}[2]$ is selected

0b'1011 - $ch_{2o}[3]$ is selected

0b'1100 - $ch_{3o}[0]$ is selected

0b'1101 - $ch_{3o}[1]$ is selected

0b'1110 - $ch_{3o}[2]$ is selected

0b'1111 - $ch_{3o}[3]$ is selected

OUT_{SELEVT2}

11:8

0

RW

OUT_{SELEVT2} select a signal that drives $events_o[2]$ from a group of 16 PWM outputs

0b'0000 - $ch_{0o}[0]$ is selected

0b'0001 - $ch_{0o}[1]$ is selected

0b'0010 - $ch_{0o}[2]$ is selected

0b'0011 - $ch_{0o}[3]$ is selected

0b'0100 - $ch_{1o}[0]$ is selected

0b'0101 - ch_{1o}[1] *isselected*

0b'0110 - ch_{1o}[2] *isselected*

0b'0111 - ch_{1o}[3] *isselected*

0b'1000 - ch_{2o}[0] *isselected*

0b'1001 - ch_{2o}[1] *isselected*

0b'1010 - ch_{2o}[2] *isselected*

0b'1011 - ch_{2o}[3] *isselected*

0b'1100 - ch_{3o}[0] *isselected*

0b'1101 - ch_{3o}[1] *isselected*

0b'1110 - ch_{3o}[2] *isselected*

0b'1111 - ch_{3o}[3] *isselected*

OUT_{SELEVT1}

7:4

0

RW

OUT_{SELEVT1} *select a signal that drives events_o[1] from a group of 16 PWM outputs*

0b'0000 - ch_{0o}[0] *isselected*

0b'0001 - ch_{0o}[1] *isselected*

0b'0010 - ch_{0o}[2] *isselected*

0b'0011 - ch_{0o}[3] *isselected*

0b'0100 - ch_{1o}[0] *isselected*

0b'0101 - ch_{1o}[1] *isselected*

0b'0110 - ch_{1o}[2] *isselected*

0b'0111 - ch_{1o}[3] *isselected*

0b'1000 - ch_{2o}[0] *isselected*

0b'1001 - ch_{2o}[1] *isselected*

0b'1010 - ch_{2o}[2] *isselected*

0b'1011 - ch_{2o}[3] *isselected*

0b'1100 - ch_{3o}[0] *isselected*

0b'1101 - ch_{3o}[1] *isselected*

0b'1110 - ch_{3o}[2] *isselected*

0b'1111 - ch_{3o}[3] *isselected*

OUT_{SELEVT0}

3:0

0

RW

OUT_{SELEVT0} *select a signal that drives events_o[0] from a group of 16 PWM outputs*

0b'0000 - ch_{0o}[0] *isselected*

0b'0001 - $ch_{0o}[1]$ *isselected*
 0b'0010 - $ch_{0o}[2]$ *isselected*
 0b'0011 - $ch_{0o}[3]$ *isselected*
 0b'0100 - $ch_{1o}[0]$ *isselected*
 0b'0101 - $ch_{1o}[1]$ *isselected*
 0b'0110 - $ch_{1o}[2]$ *isselected*
 0b'0111 - $ch_{1o}[3]$ *isselected* 0b'1000 - $ch_{2o}[0]$ *isselected*
 0b'1001 - $ch_{2o}[1]$ *isselected*
 0b'1010 - $ch_{2o}[2]$ *isselected*
 0b'1011 - $ch_{2o}[3]$ *isselected*
 0b'1100 - $ch_{3o}[0]$ *isselected*
 0b'1101 - $ch_{3o}[1]$ *isselected*
 0b'1110 - $ch_{3o}[2]$ *isselected*
 0b'1111 - $ch_{3o}[3]$ *isselected* REG_{CHEN}

Address Offset=0x104

Type: non-volatile

Field

Bits

Default Value

Access

Description

RESERVED

31:4

0

—

Reserved

CLK_{ENABLE}

3:0

0

RW

Each bit acts as HCLK clock enable for each timer.

0b'0000 - HCLK clock is disabled for all the Timers

0b'0001 - HCLK clock is enabled for Timer0

0b'0010 - HCLK clock is enabled for Timer1

0b'0011 - HCLK clock is enabled for Timer0 and Timer1

0b'0100 - HCLK clock is enabled for Timer2

0b'0101 - HCLK clock is enabled for Timer0 and Timer2

0b'0110 - HCLK clock is enabled for Timer1 and Timer2

0b'0111 - HCLK clock is enabled for Timer0, Timer1 and Timer2

0b'1000 - HCLK clock is enabled for Timer3

0b'1001 - HCLK clock is enabled for Timer0 and Timer3

0b'1010 - HCLK clock is enabled for Timer1 and Timer3

0b'1011 - HCLK clock is enabled for Timer0, Timer1 and Timer3

0b'1100 - HCLK clock is enabled for Timer2 and Timer3

0b'1101 - HCLK clock is enabled for Timer0, Timer2 and Timer3

0b'1110 - HCLK clock is enabled for Timer1, Timer2 and Timer3

0b'1111 - HCLK clock is enabled for all the Timers Firmware Guidelines Initialization:

When the HRESETn signal is low, all CSRs are set to their default value.

Four timer modules have four clock gates, which will be enabled (meaning they pass the ref clock to the respective timer module), only when either df_enable_i is high or the bit in respective position of $REG_CHENCSR$ is high (0th bit).

At every positive edge of the clock, the CSRs are updated based on APB signals.

FW can update the below bitfields to any custom value before the START bitfield in the REG_TIMx_MDCSR (where $x = 0$ to 3 for Timer0 to Timer3) CSR is set to '1' and the timer is not active yet (which means the timer is started for the first time). Other modules are commanded to be updated to the default.

The CLK_ENABLE bitfields of REG_CHEN .

The PRESC, SAWTOOTH, CLKSEL, MODE, and INSEL bitfields of REG_TIMx_FGCSR (where $x = 0$ to 3 for Timer0 to Timer3).

The $COUNT_START$ and $COUNT_END$ bitfields of REG_TIMx_HCSR (where $x = 0$ to 3 for Timer0 to Timer3).

The direction of the updown counter (default is 0)

$COMP_THRESHOLD$ and $COMP_OP$ bitfields of $REG_TIMx_H0_HCSR$ (where $x = 0$ to 3 for Timer0 to Timer3), $REG_TIMx_H1_HCSR$ (where $x = 0$ to 3 for Timer0 to Timer3), $REG_TIMx_H2_HCSR$ (where $x = 0$ to 3 for Timer0 to Timer3) and $REG_TIMx_H3_HCSR$ (where $x = 0$ to 3 for Timer0 to Timer3)

The $OUT_SEL_VT_ENABLE$, OUT_SEL_VT3 , OUT_SEL_VT2 , OUT_SEL_VT1 and OUT_SEL_VT0 bitfields of REG_EVT

PWM generation or Start the Timer:

FW can start the timer or PWM generation via the steps below.

START bitfield in the REG_TIMx_MDCSR (where $x = 0$ to 3 for Timer0 to Timer3) is set to '1' and STOP bitfield in the REG_TIMx_MDCSR (where $x = 0$ to 3 for Timer0 to Timer3) is set to '0', then all the timer and its submodules are made active.

This input signal is processed by the APB ADVANCED TIMER according to the CSR configurations. Use the $Tx_COUNTER$ bitfields in the respective $REG_TIMx_COUNTERCSR$ (where $x = 0$ to 3 for Timer0 to Timer3) to read the bit. PWM outputs are generated, which are passed to the IOMUX. Stop the Timer :

FW can stop the PWM generation. It can be done by the following steps.

START bitfield in the REG_TIMx_MDCSR (where $x = 0$ to 3 for Timer0 to Timer3) is set to '0' and STOP bitfield in the REG_TIMx_MDCSR (where $x = 0$ to 3 for Timer0 to Timer3) is set to '1', then all the timer and its submodules are made to inactive state.

The counter values will remain the same, and they will not be incremented after the Timer is stopped.

When $Tx_COUNTER$ bitfields in the respective $REG_TIMx_COUNTERCSR$ (where $x = 0$ to 3 for Timer0 to Timer3) remain

FW can update certain configurations or reinitialize the CSRs to generate a different kind of PWM. It can be done by the following steps.

START bitfield in the REG_TIMx_MDCSR (where $x = 0$ to 3 for Timer0 to Timer3) is set to '0' and STOP bitfield in the REG_TIMx_MDCSR (where $x = 0$ to 3 for Timer0 to Timer3) is set to '1', then all the timer and its submodules are made to inactive state. UPDATE bitfield in the REG_TIMx_MDCSR (where $x = 0$ to 3 for Timer0 to Timer3) is set to '1'.

Once the update to the Timer is finished, the PWM output will be holding the previous value, and $Tx_{COUNTER}$ bitfields in 0 to 3 for Timer 0 to Timer 3 will be holding the $COUNT_S$ $TART$ value. All the latest CSR configurations will be passed to the sub-modules, and once the Timer is started, it will generate a PWM output based on these configurations. Reset the Timer :

FW can reset the Timer by the following steps.

RESET bitfield in the $REG_{TIMx} MDCSR$ (where $x = 0$ to 3 for Timer 0 to Timer 3) is set to '1'.

Once the reset is issued. The PWM output will be zero, and $Tx_{COUNTER}$ bitfields in the respective $REG_{TIMx} COUNTER$ 0 to 3 for Timer 0 to Timer 3 will be holding the $COUNT_S$ $TART$ value.

Libre Software and Control

5.1 PicoRV32 Interrupt Handler

The PicoRV32 core has a built-in controller capable of handling 32 sources. The sources should be wired into the `irq` input port of the core/package. The handler also controls the `eoi` (end of interrupt) signals, which go high for the handled interrupt when the specific interrupt handler is called.

IRQs 0-2 are reserved for internal sources. These include:

1. IRQ 0 - Timer interrupt
2. IRQ 1 - EBREAK/ECALL or Illegal INstructions
3. IRQ 2 - Bus error due to unaligned memory access.

4 additional 32-bit registers (`q0-3`) are provided for IRQ handling. When the IRQ handler is called, the register `q0` contains the return address and `q1` contains the bitmask of all IRQs to be handled. This means one call to the interrupt handler needs to service more than one IRQ when more than one bit is set in `q1`. When compressed instruction mode is enabled, then the LSB of `q0` is set when the interrupt instruction is a compressed instruction. This can be used if the IRQ handler wants to decode the interrupted instruction. The other additional registers are uninitialized and scratch registers.

Custom instructions defined for interrupt handling:

1. `get rd, qs`
Copy the contents from a q-register to a general-purpose register.
2. `retirq`
Return from interrupt. As side effects, the contents of `q0` are copied to the program counter and interrupts are re-enabled.
3. `maskirq:`
There is an IRQ MASK register containing a bitmask of masked interrupts. This instruction writes a new value to the irq mask register and reads the old value.
4. `waitirq`
Pause execution until an interrupt becomes pending.
5. `timer`
Reset the timer counter to a new value. The counter counts down clock cycles and triggers the timer interrupt when transitioning from 1 to 0. Setting the counter to zero disables the timer. The old value of the counter is written to `rd`.

Part II

SoC Integration

This part is intended for audience intending to understand the platform's architecture and modify/develop their own version at an RTL level. Libre platform is composed of three subsystems:

- [Pico Subsystem](#)
- Rocket Subsystem
- Network-on-Chip (NoC) Subsystem

Pico Subsystem

The Pico subsystem (picosys) is a auxiliary subsystem organized around an AHB-Lite bus. It is used to control the rest of the SoC. The AHB-Lite bus is driven by two managers: COMMCTRL/UART Controller and PicoRV32 core package (picopkg). However, both these managers are time-multiplexed using an AHB MUX, meaning that at any time, there is only a single manager driving the bus. For additional control and interaction, the bus supports a configurable peripheral bus with options like UART, Timers, etc., on-chip SRAM memory, clock and reset generator (CRG), and a bridge to connect to the AXI port of the NoC.

Picosys uses the CRG to bring up and control the remainder of the SoC. In this chapter, we breakdown the construction and integration of the picosys in Libre-V platform:

- [PicoRV32 Core and Package](#)
- COMMCTRL
- AHB-Lite MUX and Bus
- Clock and Reset Generator (CRG)
- Peripheral Cluster
- On-chip SRAM
- AHB2AXI adapter

6.1 PicoRV32 Core and Package

On Chip SRAM

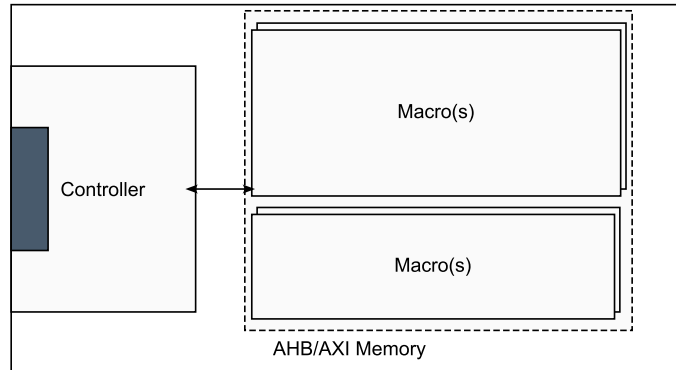


Figure 7.1: AXI/AHB SRAM module general block view.

7.1 Overview

SRAM macros usually expose a simple interface. Such an interface is seldom compatible with the port interfaces exposed by Network on Chip (NoC) or Network Interconnects (NIC). These systems employ elaborate on-chip interconnect standards like AHB/AXI¹ as they need to handle multiple transactions from multiple devices concurrently. Such protocols include transaction-related metadata signals which enable this capability. We require a controller frontend that will consume these highly informative AHB/AXI transactions and translate it into low-level SRAM interface access(es). We first provide a brief overview of the common controller logic for each of the protocol and then conclude with a description of the generator utility provided. Figure 7.1 shows the common block view of the SRAM module. The macros are instantiated independently of the controller via rule checking if using the optional configurator. Named `gen_lib_sram_*`, the user is expected to replace these prior to synthesis and/or pd with actual models from the foundry.

7.2 AHB Controller

The AHB family of macros uses `ahb2sram`, a 32-bit² AHB-Lite subordinate for a single-port synchronous SRAM. This controller module is parameterized with the geometry of the module the user expects. This can be different from the configuration of SRAM macros available to the user. The optional generator tool, discussed later, is capable to combining available macros to match the user's specific requirements.

Controller Parameters:

1. `LINE_W`: required SRAM line width in bits
2. `LINE_AW`: use to define the depth ($2^{\text{LINE_AW}}$) of the SRAM in bits

¹None of the AMBA Bus protocols will be discussed in detailed here. Please refer to the **official** documentation(s) available online.

²At the time of writing this, the controller only supports and is tested for 32-bit word size.

3. WORDS: number of 32-bit words. The current controller is only capable of supporting 32-bit words. Hence, this parameter should always equal $\text{LINE_W}/32$.

7.2.1 AHB-Lite Primer

An AHB-Lite transfer is split across two pipelined phases - address phase (aphase) and data phase (dphase). In the aphase the manager drives HADDR, HTRANS, HWRITE, and HSIZE, qualified by HSEL and the shared HREADY. In the subsequent dphase, write data (HWDATA) or read data (HRDATA) is exchanged, with the subordinate controlling the bus through its registered HREADYOUT and reporting status on HRESP. HTRANS[1] distinguishes an active (NONSEQ/SEQ) beat from IDLE/BUSY. Our implementation does not support burst-type transfers. This is in align with the behavior of the PicoRV32 core's native memory interface.

7.2.2 Phase Sequencing and Timing

The least significant bits of the address are word alignment bits and will be always zero for word size greater than 8 b. The next significant bit(s) are used for word select for wide SRAM lines. An active address beat is registered on $\text{aphase} = \text{HSEL} \ \& \ \text{HREADY} \ \& \ \text{HTRANS}[1]$. A read drives the SRAM in the same cycle and its data is returned in the data phase; a write is held one cycle so that its data phase supplies HWDATA. Consequently a read incurs no wait state and a write incurs exactly one.

The chip enable, global write enable, and address to the macro are

$$\text{CEN} = \sim (\text{read_en} \mid \text{write_en_reg}), \quad \text{GWEN} = \text{read_en}, \quad \text{A} = \text{read_en} ? \text{line_addr_nxt} : \text{line_addr_reg},$$

so a read presents the incoming address immediately while a write replays the registered one.

7.2.3 Byte Lanes and Write Enables

From HSIZE and the lower address bits the controller derives a one-hot-per-byte lane mask (byte, half-word, or full word), registers it into the data phase, and expands it into the macro's active-low, bitwise write enable. When a line is wider and can accomodate several bus-words ($\text{WORDS} > 1$) the addressed 32-bit word is selected within the line for both the write mask and the read data; otherwise the line is the word. On a read the registered byte lanes gate HRDATA; HRESP is tied to OKAY.

7.3 AXI Controller

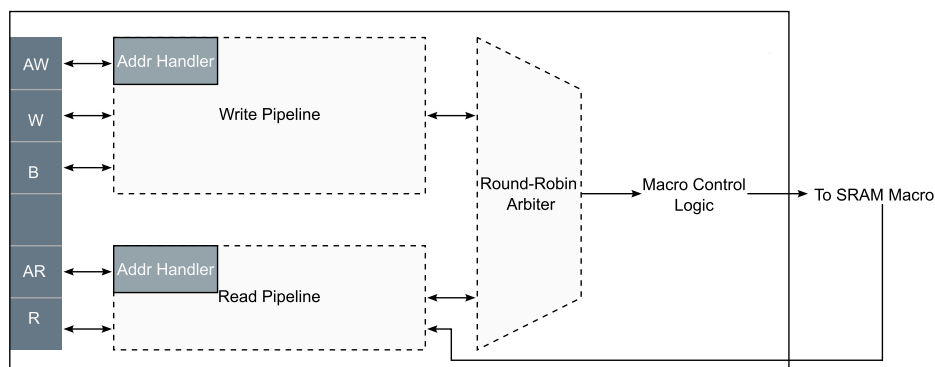


Figure 7.2: axi2sram controller detailed block view.

Similar to the AHB controller, there is an AXI controller (axi2sram) that converts AXI transactions to SRAM control logic. However, AXI is a more complex (and thereby capable) protocol compared to AHB. Figure [Figure 7.2](#) provides an overview of the architecture of this controller. Note that the dotted lines are

only meant to represent sections of the logic and are not implemented as independent sub-modules. Only the address handler module that is shared by the both pipelines is implemented separately as a module.

7.3.1 AXI Addressing Primer

Most of the understanding about AXI addressing has to do with Burst transactions. Burst transactions involve multiple transfers. In these, the manager is expected to provide only a start address and the subordinate is expected to handle the rest of the transfers based on that address and some control signals.

There are two key control signals:

1. **AxSIZE**: This signal carries information about the number of bytes of information accessed in each transfer (also called a beat) within a Burst transaction. Specifically,

$$\text{Number of bytes} = 2^{\text{AxSIZE}}.$$

The width of this signal and the value it can assume is constrained by the data width. AXI does not permit packet sizes greater than size of the Data bus. The width can be calculated as follows:

$$\text{Width} = \lceil \log_2 \left(\frac{\text{DW}}{8} \right) \rceil.$$

($\lceil \log_2 \rceil$ rounds up; for a power-of-two DW/8 it is exact.)

2. **AxLEN [7:0]**: This indicates the number of transfers to be completed within a single Burst transaction. Specifically,

$$\text{Number of transfers} = 1 + \text{AxLEN}$$

Here Ax stands for AW or AR.

There are three types of Burst transactions possible:

- **FIXED**: In a Burst type of FIXED type, same address is used repeatedly. An example application provided in the AXI specification is for emptying a FIFO. Generally, used very little.
- **INCR**: INCR Burst transactions are more common and align with the intuitive picture of a burst transfer. It sequentially increments the address until specified.
- **WRAP**: A WRAP Burst transaction is similar to INCR, except it wraps around at a specific boundary depending on transaction attributes.

AXI “Regular” transactions impose certain restrictions on the attributes of the transactions. See section A3.1.8 of AMBA IHI0022 document. Of those, our address generation submodule (`axi_addr_handler`) assumes the following:

1. Number of transfers: INCR bursts of 1–256 beats; WRAP bursts of {2, 4, 8, 16} beats.
2. Size is the same as the data channel if Length is greater than 1.
3. Address is aligned to the transaction container for INCR transactions.
4. Address is aligned to Size for WRAP transactions.
5. Transactions are not permitted to cross a 4KB boundary (section A3.1.3, AMBA IHI0022)

7.3.2 Implementation

The AXI memory module consists of the following components: Main controller logic for handling AXI transactions, a submodule for handling burst addressing, and the native interfaced, SRAM macro. Here only discuss the controller. The SRAM macro exposes a read port (Q), write port (D), clock (CLK), chip enable port (CEN), global write enable (GWEN), local write enable (WEN), address port (A), and additional signals³, STOV, EMA, EMAS, EMAW.

³These are ignored for simulation purposes.

1. Handling the addressing:

The module `axi_addr_handler` is responsible for handling the addressing during Burst transactions. Given a start address, type of burst transfer, size of each transfer, and length of transaction, it generates addresses for each beat of the transaction. It is implemented as a two state FSM: IDLE and ACTIVE. It accepts new start address (called descriptor in the source code) whenever it is in IDLE state. This is indicated to the external environment by asserting the `start_addr_ready` signal. Once the external module asserts `start_addr_valid` signal, this module samples all the input signals, registers them, and enters ACTIVE state. Once in ACTIVE, it uses a count-down timer to track each beat. On the last count, the `is_last_addr` flag is asserted and the FSM moves into IDLE state. The module uses the `next_addr_ready/valid` handshake to move to the next address in the Burst transaction.

The next address for Burst transactions is calculated as follows:

$$\text{next_addr} = \begin{cases} \text{curr_addr} & \text{if AxBURST} = \text{'FIXED'} \\ \text{curr_addr} + 2^{\text{AxSIZE}} & \text{if AxBURST} = \text{'INCR or within wrap boundary when AxBURST} = \text{'WRAP'} \\ \text{wrap_base} & \text{if AxBURST} = \text{'WRAP and crossing wrap boundary.} \end{cases} \quad (7.1)$$

Here, `wrap_base` is the aligned base of the wrap region. Refer to the AXI specification for how this might be calculated⁴. This module assumes that the start address is aligned to Size and one increment indicates a word-size (element) increment of 2^{AxSIZE} bytes, i.e., the byte address will be incremented by 4 if Size = 4 bytes. Cache accesses is the main motivation of WRAP Bursts.

AXI4 limits the length of WRAP transactions to 2/4/8/16. This provides a neat little trick used to calculate the next address in a Wrap transaction without using the modulo operator⁵⁶. For these Lengths, the value of `AxLEN` is 0001, 0011, 0111, 1111, respectively. Notice that once the address has been adjusted for word alignment (i.e., those least significant zeros are shifted out), then only the bit positions where there is a 1 in value of `AxLEN` should be changed in the address (when `AxLEN` is aligned with least significant end of the address. Rest of the address should carry over from the before. This trick is realized with the following equation:

$$\text{next_addr_wrap} = (\text{wrap_mask} \& \text{next_addr_incr}) \mid (\sim \text{wrap_mask} \& \text{curr_addr}). \quad (7.2)$$

In the above expression, most of the terms should be self-explanatory, except perhaps `wrap_mask`. This term is simply the `AxLEN` signal shifted to account for the word alignment and padded for address size. Bitwise complement of wrap mask has 1 in bit positions that should not change and be carried over from current address.

2. Main Controller Logic

The main controller is implemented to try and mimic the behavior of `IntMemAxi` IP from ARM, which is a licensed IP that cannot be open-sourced. However, its functionality is described and can be found [here](#). However, we cannot guarantee the identical behavior as our development was independent of this IP.

This module is designed for controlling a single-port SRAM module. The reads and write paths are implemented independently and arbitrated in a round-robin fashion with reads being the default mode. Controller Parameters

- (a) DW: Word Size (only supports 32/64-bit)
- (b) IDW: AXI ID signals width
- (c) LINE_W: required SRAM line width
- (d) LINE_AW: use to define the depth of SRAM (in bits)
- (e) WORDS: Number of words per SRAM line
- (f) READ_WAIT: (0/1) Number of wait stages for reads.

⁴WRAP Burst is intended for cache implementations, where, to minimize stall penalty, the first word is delivered immediately and the rest of the cache line is then fetched sequentially (wrapping).

⁵Credits for this trick: <https://zipcpu.com/blog/2019/04/27/axi-addr.html>.

⁶My instinct is that they limit the length for Wrap Bursts for this very reason. Or it may just be that 16 is sufficient for most applications.

This controller can support multiple words per SRAM line. Further, the reads can have an optional wait stage that allows pipelining. Note that like the AHB controller, the WORDS parameter should be sensible and will depend on the word size and the line width.

The write path is relatively straightforward. Whenever a new write request arrives, the address handler is used to unravel and stream the addresses in the burst sequence. Once available and the previous transaction is completed, a new request write request is raised.

The read path is slightly more complicated by the optional wait state. Whenever a new read request is granted by the arbiter, the read path becomes busy and the count down timer is topped up. Once the count down is complete, the read value appears on the read port of the macro and is subsequently transferred to RDATA bus when it is available.

Both, read and write, paths are pipelined to improve performance. Further, similar to AHB controller, rest of the logic is used to generate the line-wide masks by extending the Strobe signals and word select bits.

7.4 Libremc

Terminology: We use “module” refers to the entity that actually connects to a bus or a network on chip. The user specifies the module geometry. “Macro” is used to refer to the actual SRAM provided by the foundry/memory compilers. Such macros have a primitive interface and might have limited configurations.

We provide a support utility called Libremc (short for Libre macro compiler) that enables the SoC configurator to easily build SRAM modules for connecting to either Pico subsystem or the NoC Subsystem. Consequently, it can configure the

1. Frontend protocol: Either AXI or AHB.
2. Width (W): width of the SRAM module.
3. Depth (D): depth of the SRAM module.

The word size is fixed to 32 bits for AHB slave memories and to 64 bits for AXI ones⁷. Further, the address bus width is fixed to 32 bits.

However, often when synthesizing memory modules, one might not have access to the SRAM compilers or the available compilers might have limited capability and only a few macro configurations are accessible. Libremc allows the user to specify the available macro configurations in a .conf file as follows:

#	Depth	Width	DepthStride	WidthStride	Mask-Granularity
16384	32	0	0	8	
16384	64	0	0	8	
4096	64	0	0	8	
8192	64	0	0	8	
512-8192	32	512	0	8	

The values can be specified explicitly or as (low - high) ranges.

The tool uses this information to find the “optimal” combination to meet the user specified module requirements. To keep things simple, we solve a simple grid problem. For each available macro $m_i = (w_i, d_i)$ configuration, the following cost function f_i is evaluated:

$$f_i = c_i \times r_i \times a_i, \quad (7.3)$$

⁷This is because the bus specifications are a part of the architecture which is common to the platform.

where, $c_i = \text{ceil}(W/w_i)$, $r_i = \text{ceil}(D/d_i)$, and a_i is the area of macro. When unavailable, the area of the macro can be estimated as the product of a macro's width w_i and depth d_i ⁸.

The tool selects the configuration as follows:

1. Configuration with the lowest value of cost function.
2. Any ties are broken by choosing the configuration with lowest number of instances ($r_i \times c_i$).
3. Remaining ties are broken by choosing the configuration using the smallest macro.

Once a configuration is found, the tool then automatically generates the module RTL along with a configuration file specifying the chosen macro and grid parameters. In this generated SRAM module it instantiates the requested controller and the grid of macros. In addition, it outputs the word select logic.

7.4.1 Usage

1. Generate a memory module:

```
$ make gen PROTO=<ahb|axi> SIZE=<lines>x<width> [MACROLIST=<file>.conf]
```

2. Elaborate and test a generated module:

```
$ make test-one PROTO=<ahb|axi> SIZE=<lines>x<width> [EXTRA_AW=n]
```

3. Generate and test a memory module in one step:

```
$ make test PROTO=<ahb|axi> SIZE=<lines>x<width> [MACROLIST=<file>] [EXTRA_AW=n]
```

4. Clean up generated files:

```
$ make clean
```

7.5 Summary

While the AHB controller logic was mostly inherited from Base-SoC (ARM), the AXI controller was developed in-house. AXI's Internal memory interface⁹ behavior combined with SRAM Macro interface were used as a reference. We improved upon the IntMemAxI module¹⁰ by directly supporting AXI4 (it supported AXI3) and directly generating the SRAM interface.

Testing the Libremc utility is a non-trivial exercise and in fact a tedious one. Hence, we would appreciate any feedback on the tool, bug finds/fixes in the source code, and inaccuracies in this document.

⁸This does not capture the wiring overhead and may not correlate well with the actual area. We are working to include this wiring overhead to improve this tool.

⁹<https://support.arm.com/documentation/dto0009/a/about-the-internal-memory-interface?lang=en>

¹⁰We abide by the licensing terms by not using the source code for development of our own version. The functionality has been independently replicated.